



RAS COUNCIL (2025-26)
IIT MADRAS



SKILL DEVELOPMENT CELL-RESEARCH WORKSHOP ON

Leveraging HPC for Modern Scientific & Engineering Research

SPEAKERS



Prof. Phani Kumar

Professor
Department of Metallurgical and
Materials Engineering, IIT Madras



2.00 PM TO 3.30 PM

Overview of high-performance computing
Importance and relevance of HPC in modern
engineering and scientific research
Basic workflow for job submission and
resource usage
Data handling guidelines and best practices



**Dr. Nagabhushana
Rao Vadlamani**

Associate Professor
Department of Aerospace
Engineering, IIT Madras



3.45 PM TO 5.00 PM

Applications of HPC in Aerospace
Computational challenges and methodologies
Guidance for students integrating HPC into
their research



13 DECEMBER 2025



2.00 PM TO 5.00 PM



RJN 101

SCAN TO REGISTER



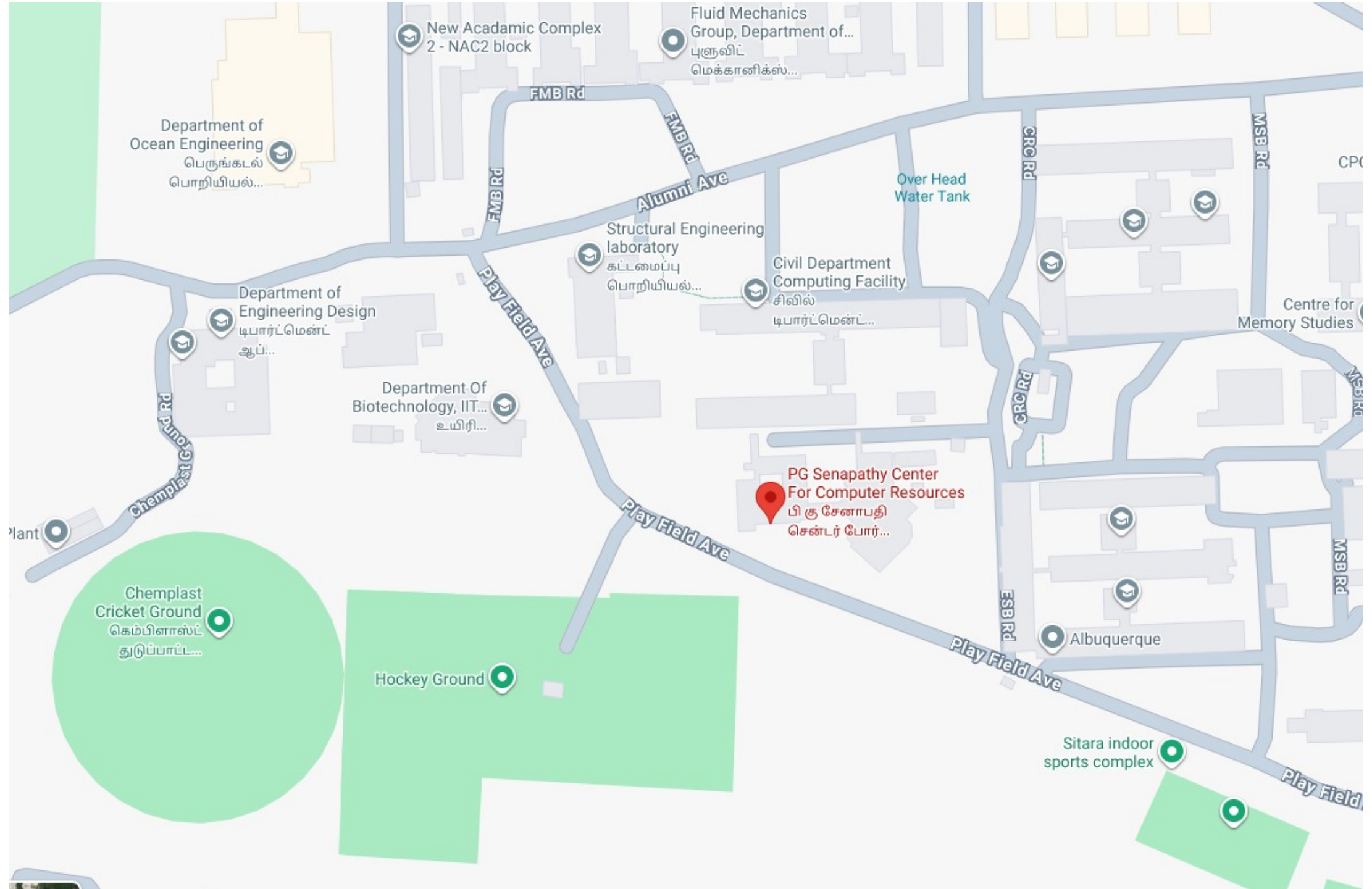
Introduction to High Performance Computing

Gandham Phanikumar

Head, Computer Center
&

Professor, Department of Metallurgical and Materials Engineering
Indian Institute of Technology Madras

Skill Development Cell - Research
Workshop on
Leveraging HPC for Modern Scientific & Engineering Research,
Research Scholars' Day 2025, 13-Dec-2025 @RJN 101



CC before Nov-2025



கணினி மையம்

कंप्यूटर केंद्र

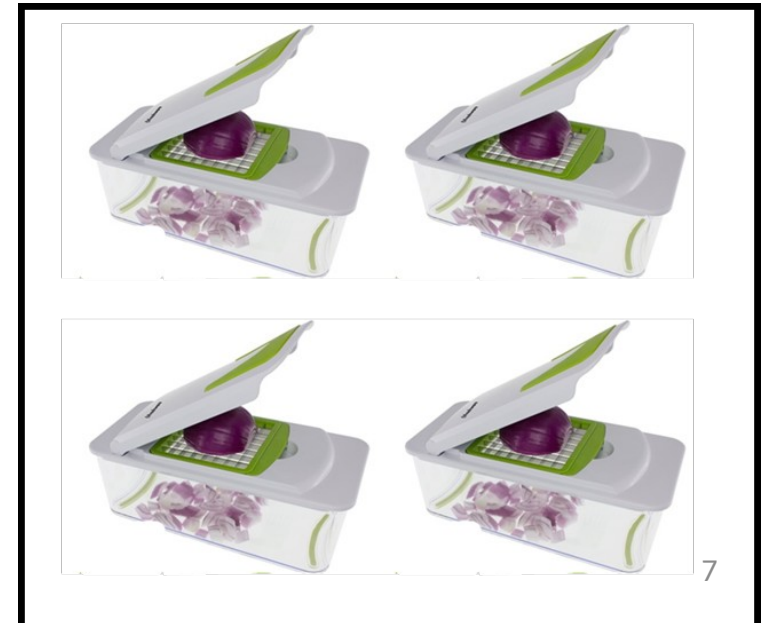
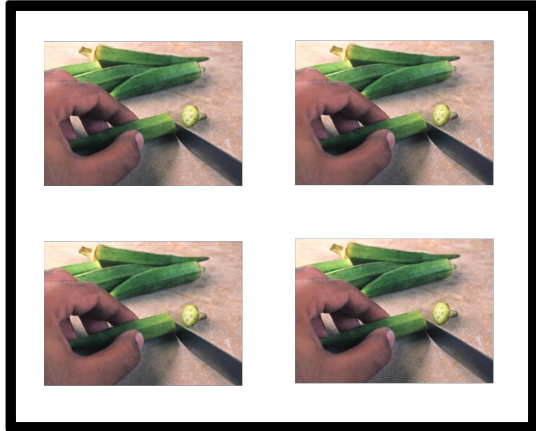
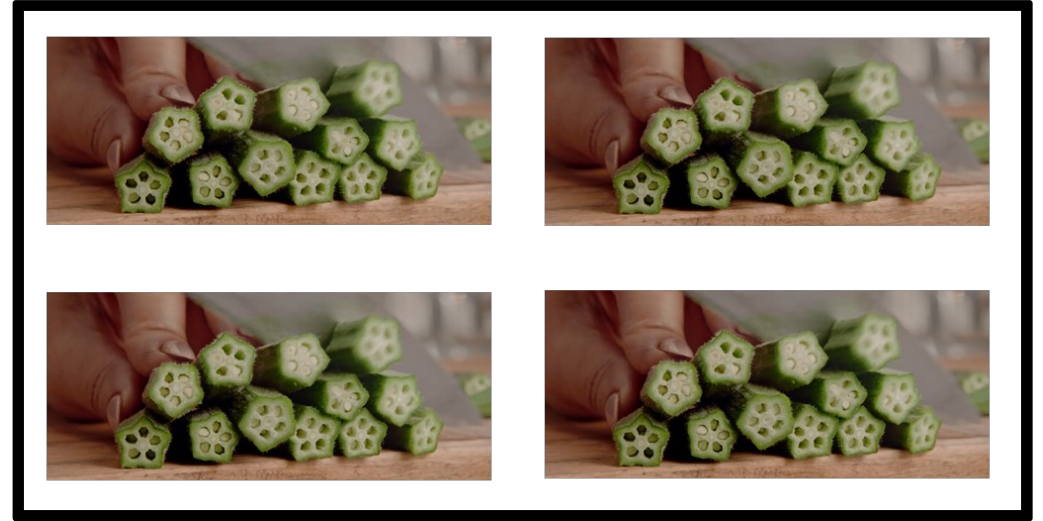
COMPUTER CENTRE

P.G. SENAPATHY
CENTRE FOR
COMPUTING RESOURCES
PARAM SHAKTI

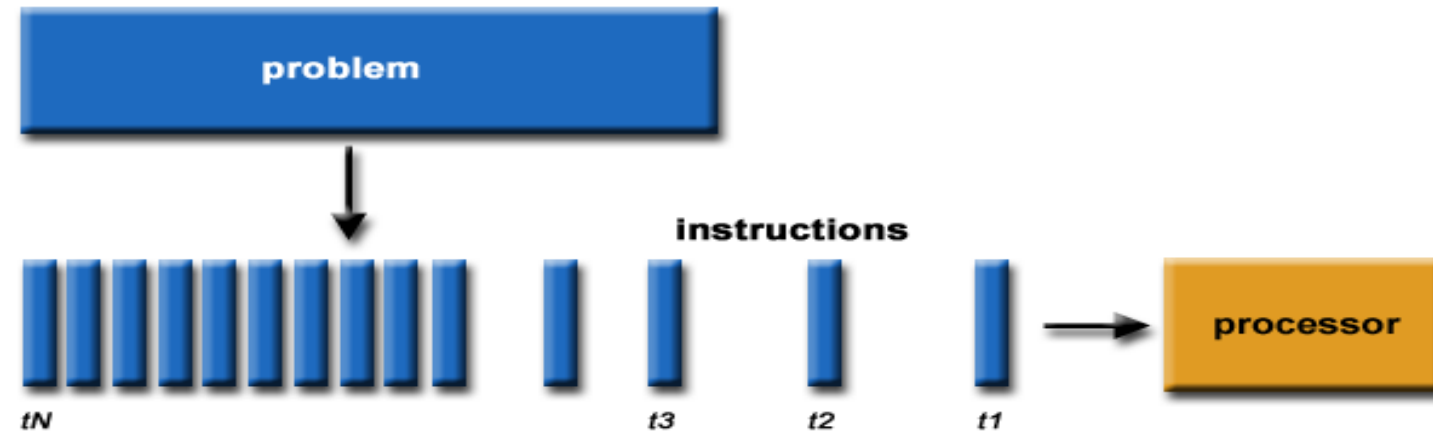
Outline

- Overview of high-performance computing
- Importance and relevance of HPC in modern engineering and scientific research
- Basic workflow for job submission and resource usage
- Data handling guidelines and best practices

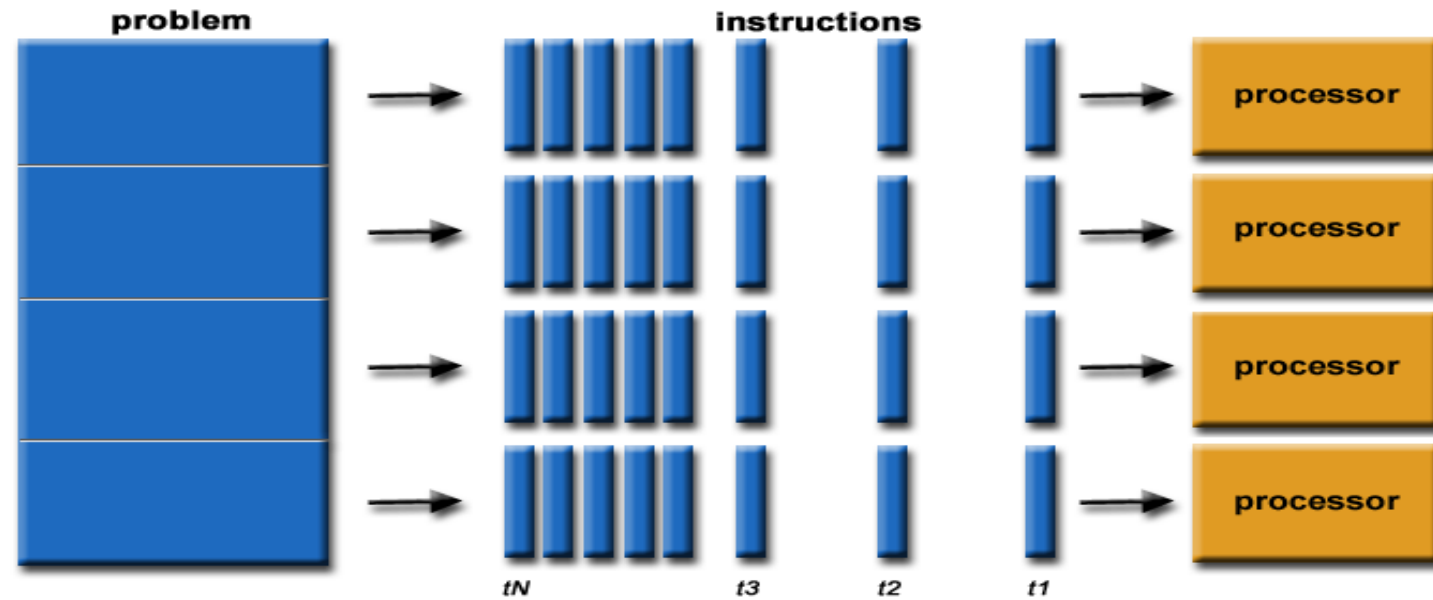
High Performance = to be faster at repetitive tasks !!



Serial Computing



Parallel Computing



Parallel Computers

- Data level parallelism
- Instruction-level parallelism
- Thread-level parallelism
- Process-level parallelism

Parallel thinking is an acquired skill

Multicore single processors are already parallel computers



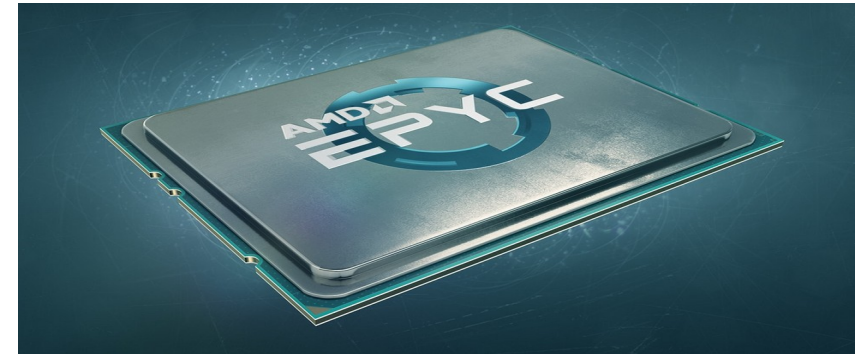
Intel Core i9-12900K Processor;
24 Cores; 48 Threads; 2.4 Ghz; upto 6 GHz



Intel Xeon 6 Processor 6978P
120 Cores; 240 Threads; 2.1 Ghz; upto 3.9 GHz



AMD RYZEN Threadripper PRO 9995 Processor
96 Cores; 192 Threads; 2.4 Ghz; upto 5.4 GHz



AMD EPYC 9965 Processor
192 Cores; 384 Threads; 2.25 GHz; upto 3.35 GHz

Multiple sockets

Multiple CPUs on same motherboard



IBM Power E1180
Full Rack 16 Socket system



HPE Superdome Flex

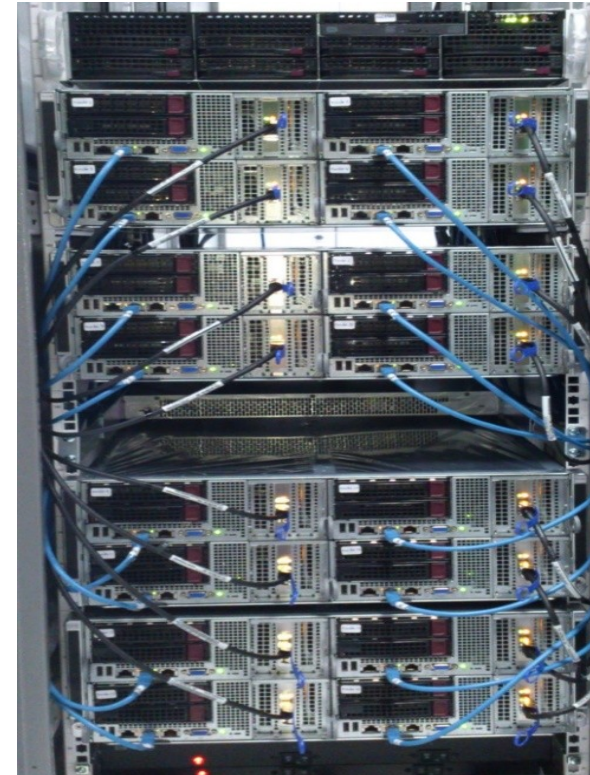
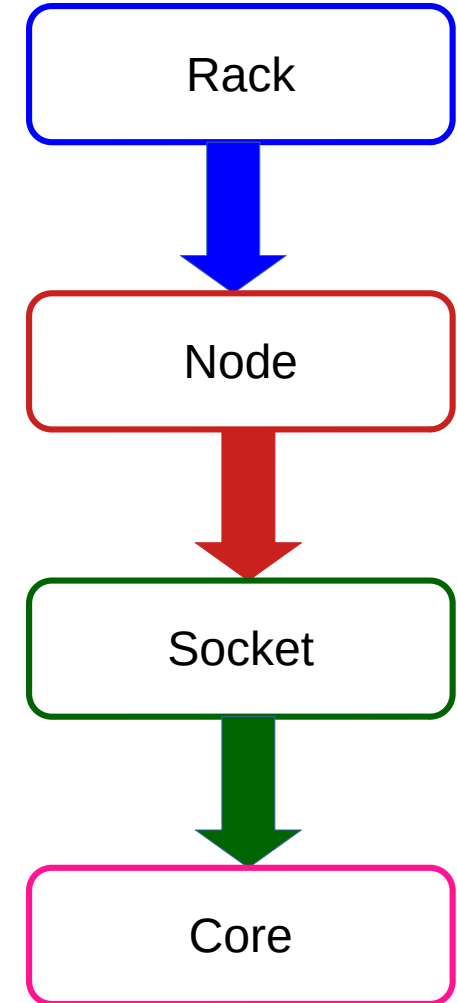
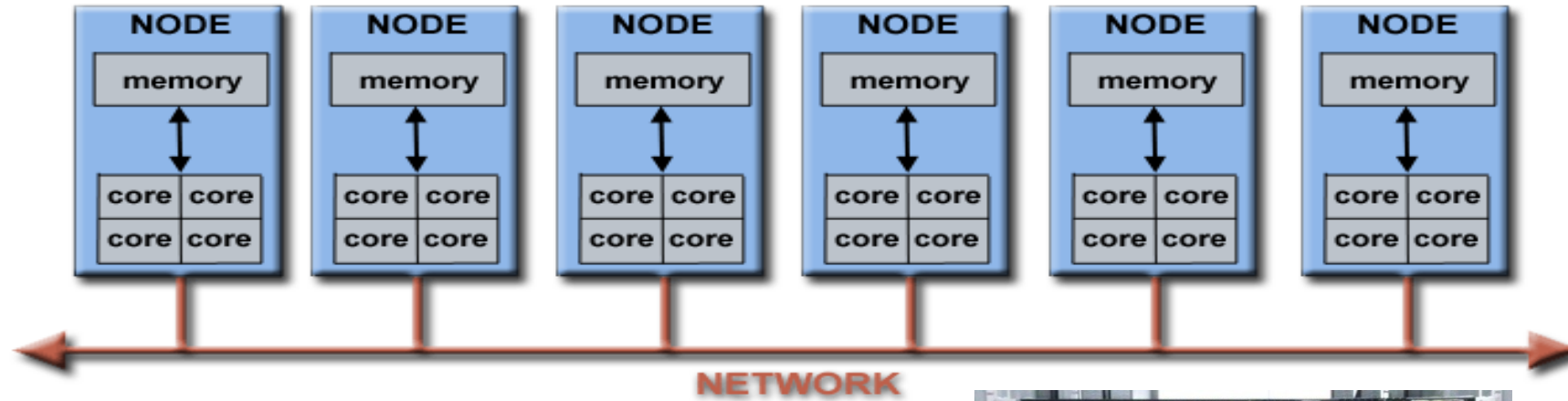
A single system with 4-32 sockets and 1-48 TB of in-memory computing capacity into help you solve complex, data-intensive HPC problems at unparalleled scale.



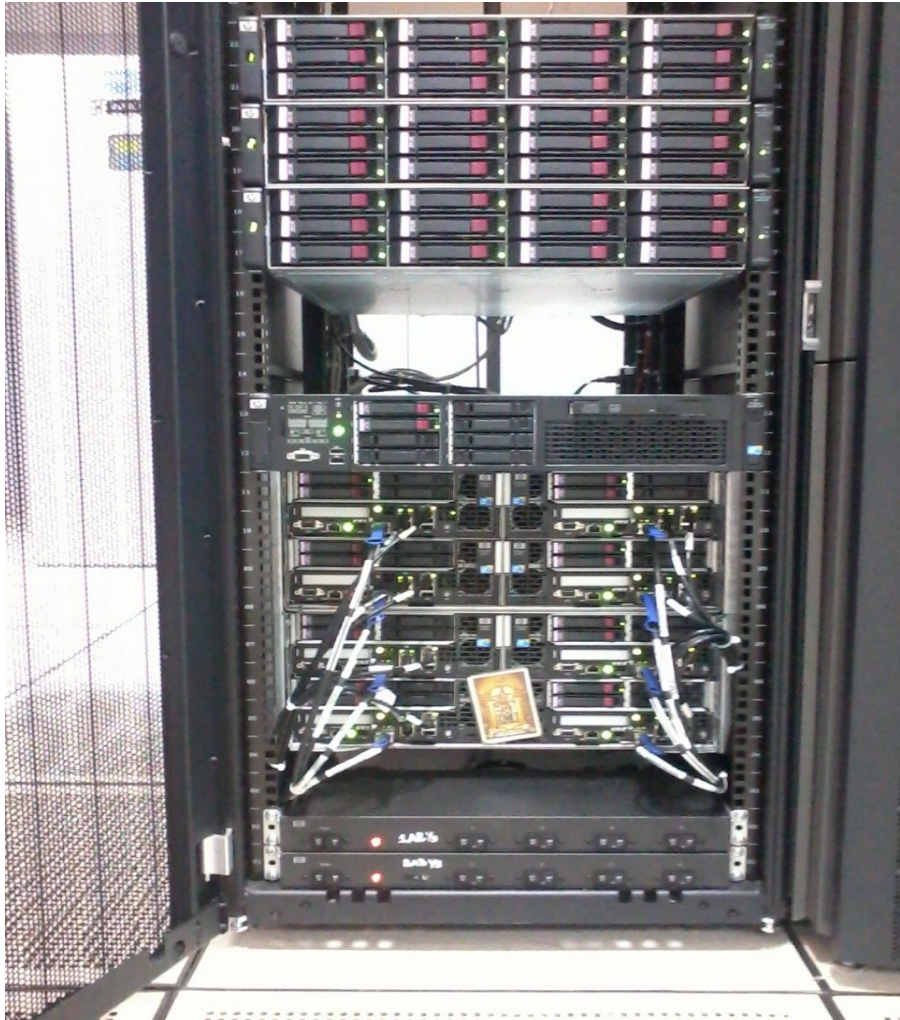
SuperMicro SYS-681E-TR
6U 8-Socket system

SMP systems allow for automatic parallelization by compilers

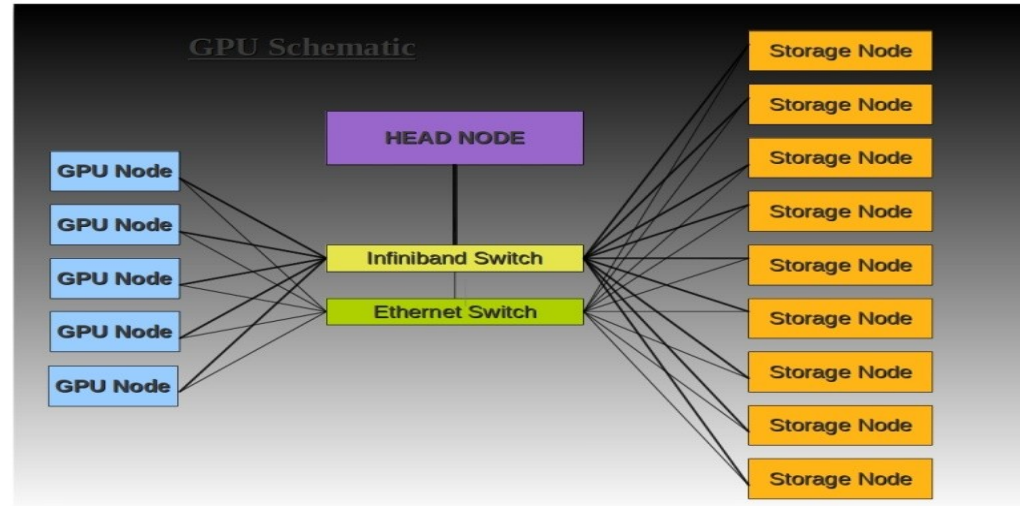
Networked computers are parallel computers



Typical look of a rack



Example: Libra Cluster at P.G.Senapathy
Centre for Computing Resources, IIT Madras



GPUs are parallel computers



NVidia H200 SXM GPU

141 GB Memory
67 Teraflops in FP32
34 Teraflops in FP64
4.8 TB/s memory bandwidth
3958 TFLOPS FP8 Tensor Core

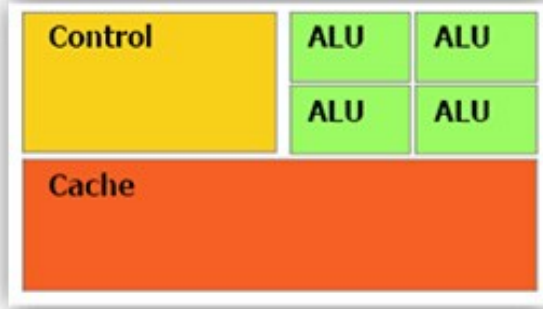


AMD Instinct MI350 GPU

128 GB Memory
144.2 TFLOPS in FP32
72.1 TFLOPS in FP64
8 TB/s peak memory bandwidth
9.2 PFLOPS OCP-FP8

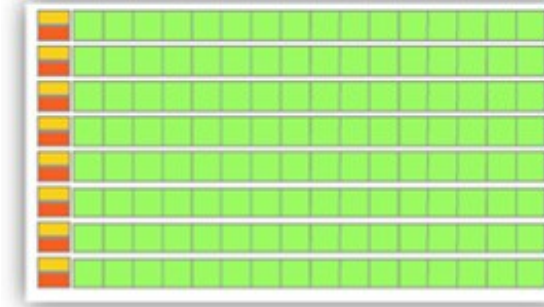
CPU vs GPU

CPU



- Low compute density
- Large caches
- Complex control logic
- Optimized for serial operations
 - Fewer execution units
 - Higher clock speeds
- Newer CPUs have more parallelism

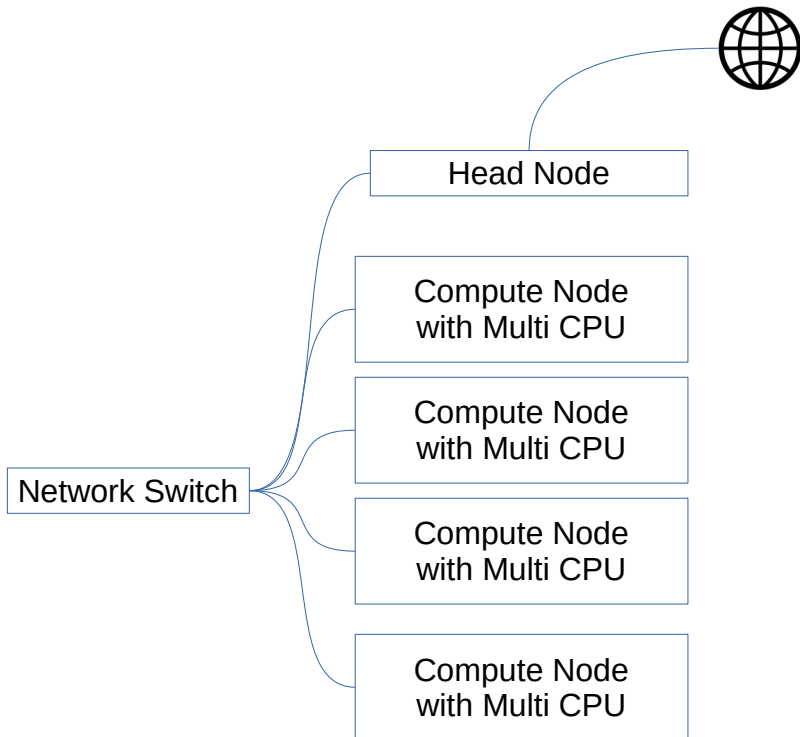
GPU



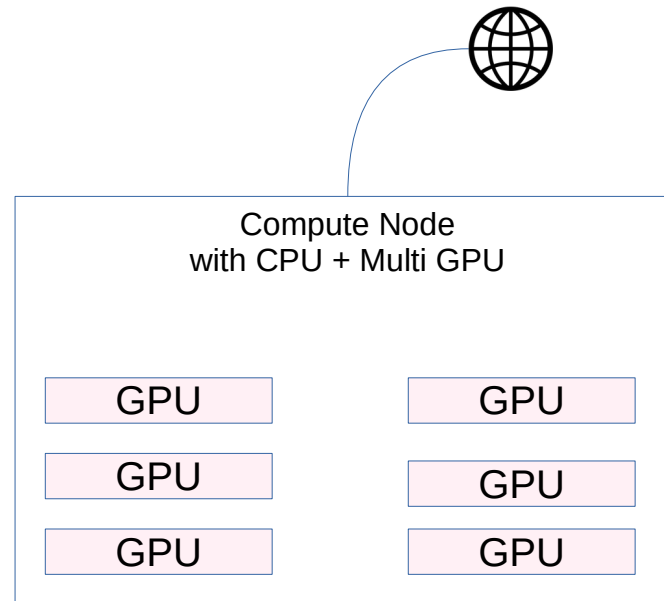
- High compute density
- Small caches
- High computations per memory access
- Built for parallel operations
 - Many execution units (ALUs)
- High throughput

HPC Architectures

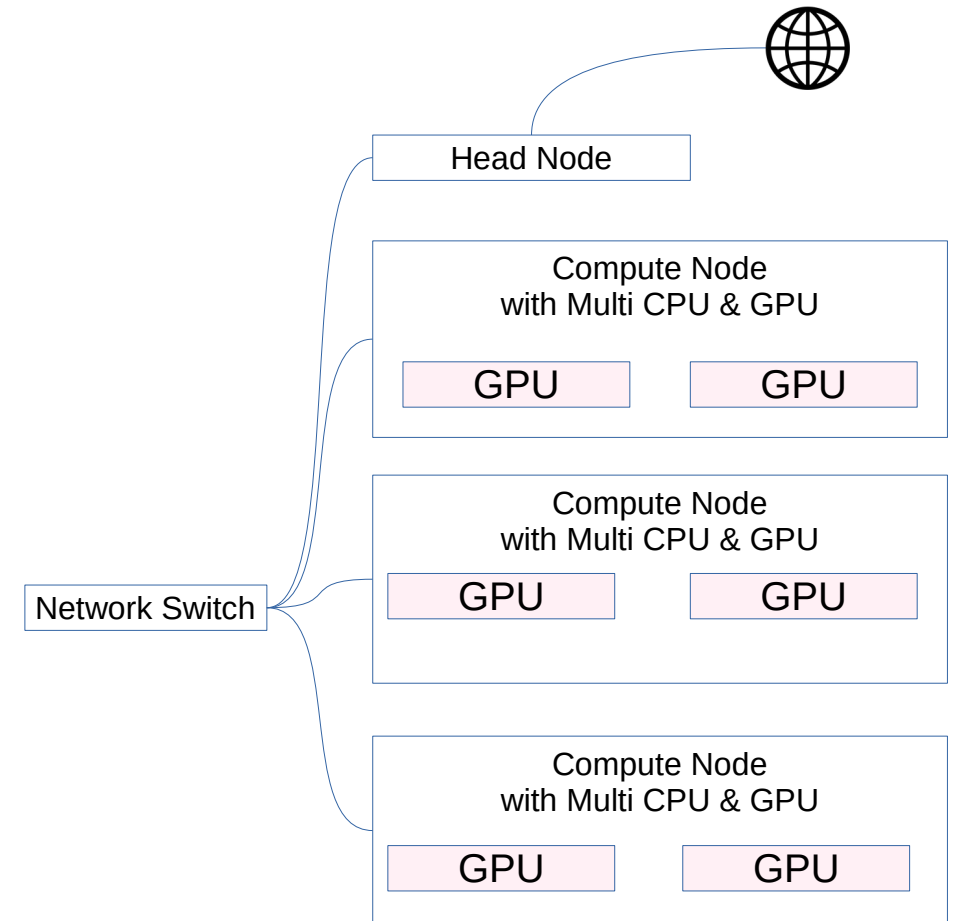
CPU Cluster



GPU Cluster



Heterogeneous Cluster



HPC Clusters

National Supercomputing Mission

- <https://nsmindia.in/>
- <https://dst.gov.in/national-super-computing-mission>
- <https://nsm.iitm.ac.in/>
- <https://cc.iitm.ac.in/> → ParamShakti Cluster

Paramshakti at IIT Madras



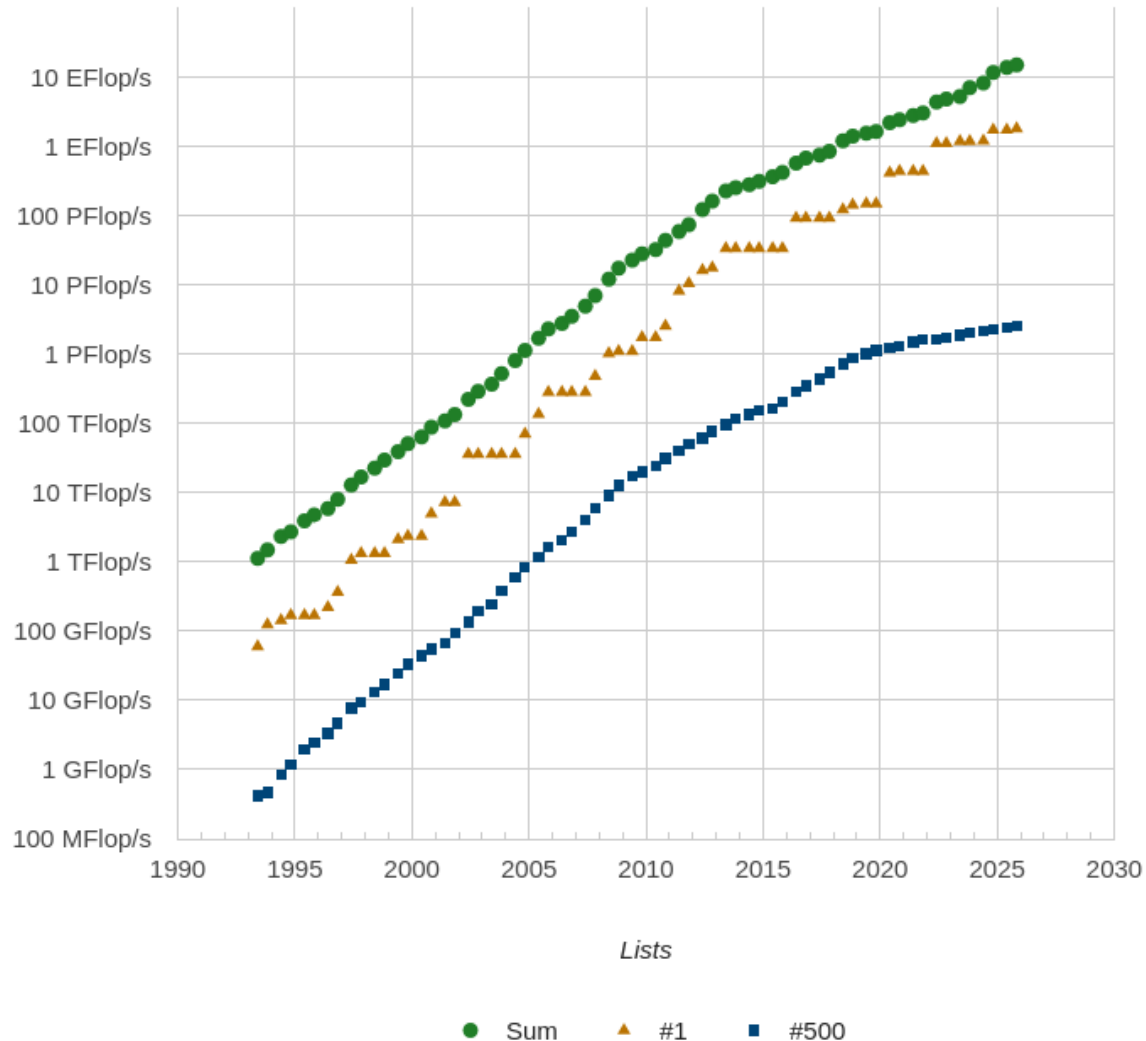
- 622 nodes, 96 with 768 GB RAM
- 28,896 cores
- 30 nodes with 2 Nvidia A100 GPUs each
- 4 PB storage @40 GB/s
- 400 TB storage @100 GB/s
- 3.1 PFLOPS



Power consumption: 0.5 MW

Statistics from top500.org as in Jun-2025

Performance Development



Fastest Supercomputer in the world

**El Capitan - HPE Cray EX255a, AMD
4th Gen EPYC 24C 1.8GHz, AMD
Instinct MI300A, Slingshot-11, TOSS**

**11,039,616 cores
AMD 4th Gen EPYC 24C 1.8 GHz
Rpeak: 2,746 Pflop/s
Power: 29,581 kW
Interconnect: Slingshot-11**

**Lawrence Livermore National Lab,
USA**

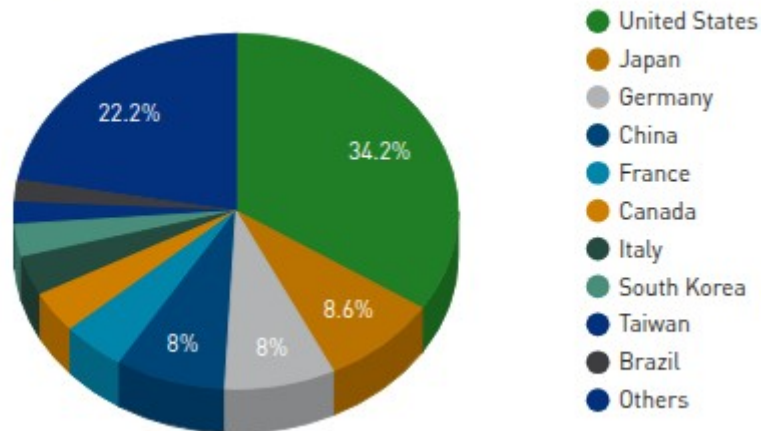
Statistics from top500.org as on Nov-2025

Common Feature of top ranked supercomputers:

They are hybrid systems
(Multiprocessor + Accelerator)
running Linux

Rank	System	Cores	Rmax (PFlop/s)	Rpeak (PFlop/s)	Power (kW)
1	El Capitan - HPE Cray EX255a, AMD 4th Gen EPYC 24C 1.8GHz, AMD Instinct MI300A, Slingshot-11, TOSS, HPE DOE/NNSA/LLNL United States	11,340,000	1,809.00	2,821.10	29,685
2	Frontier - HPE Cray EX235a, AMD Optimized 3rd Generation EPYC 64C 2GHz, AMD Instinct MI250X, Slingshot-11, HPE Cray OS, HPE DOE/SC/Oak Ridge National Laboratory United States	9,066,176	1,353.00	2,055.72	24,607
3	Aurora - HPE Cray EX - Intel Exascale Compute Blade, Xeon CPU Max 9470 52C 2.4GHz, Intel Data Center GPU Max, Slingshot-11, Intel DOE/SC/Argonne National Laboratory United States	9,264,128	1,012.00	1,980.01	38,698
4	JUPITER Booster - BullSequana XH3000, GH Superchip 72C 3GHz, NVIDIA GH200 Superchip, Quad-Rail NVIDIA InfiniBand NDR200, RedHat Enterprise Linux, EVIDEN EuroHPC/FZJ Germany	4,801,344	1,000.00	1,226.28	15,794
5	Eagle - Microsoft NDv5, Xeon Platinum 8480C 48C 2GHz, NVIDIA H100, NVIDIA Infiniband NDR, Microsoft Azure Microsoft Azure United States	2,073,600	561.20	846.84	
6	HPC6 - HPE Cray EX235a, AMD Optimized 3rd Generation EPYC 64C 2GHz, AMD Instinct MI250X, Slingshot-11, RHEL 8.9, HPE Eni S.p.A. Italy	3,143,520	477.90	606.97	8,461
7	Supercomputer Fugaku - Supercomputer Fugaku, A64FX 48C 2.2GHz, Tofu interconnect D, Fujitsu RIKEN Center for Computational Science Japan	7,630,848	442.01	537.21	29,899

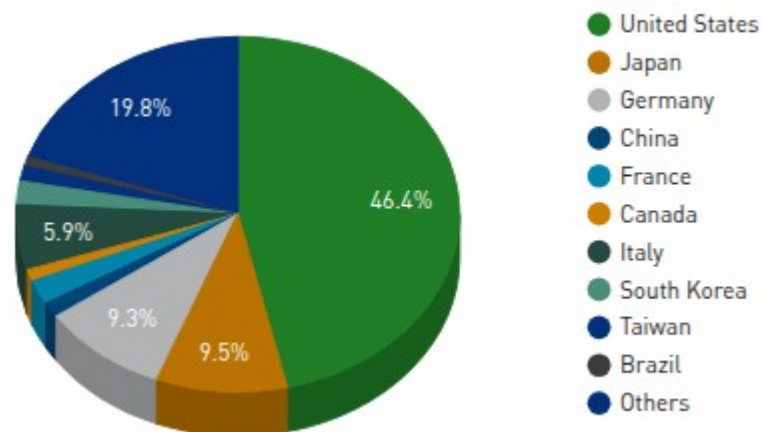
Countries System Share



India #17 @ 1.2% of total HPC in the world



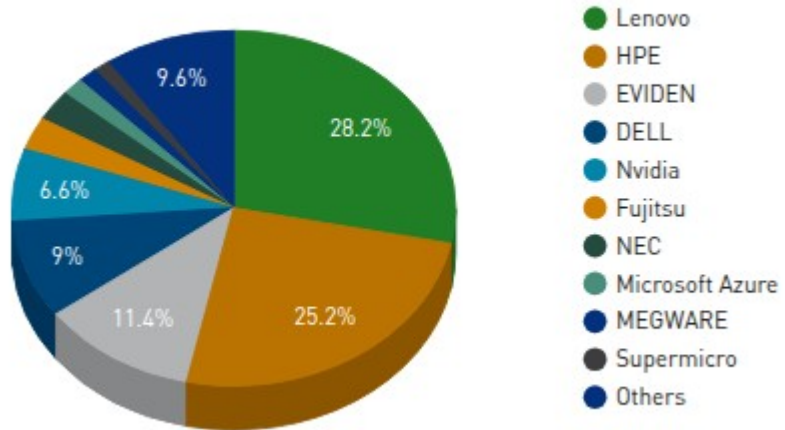
Countries Performance Share



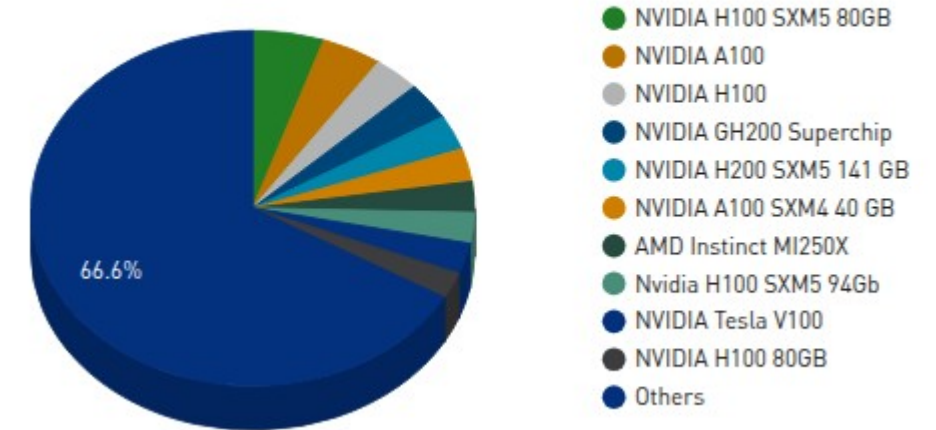
	Countries	Count	System Share (%)	Rmax (GFlops)	Rpeak (GFlops)	Cores
1	United States	171	34.2	6,961,330,490	11,002,146,951	58,653,128
2	Japan	43	8.6	1,425,992,450	1,856,582,938	13,615,160
3	Germany	40	8	1,396,125,580	1,821,152,384	8,831,292
4	China	40	8	204,933,614	343,819,382	13,617,624
5	France	23	4.6	334,639,840	514,248,244	4,493,824
6	Canada	19	3.8	158,216,260	242,003,075	1,990,192
7	Italy	18	3.6	888,816,920	1,149,785,108	7,716,656
8	South Korea	15	3	323,114,900	442,276,740	2,799,412
9	Taiwan	10	2	208,612,520	300,088,670	1,063,168
10	Brazil	10	2	124,511,650	242,547,142	1,158,880
11	Norway	9	1.8	44,075,210	62,065,550	642,528
12	United Kingdom	9	1.8	378,120,110	522,317,268	2,548,776
13	Poland	8	1.6	70,980,670	110,451,520	440,000
14	Sweden	8	1.6	66,959,520	95,140,566	451,200
15	Saudi Arabia	7	1.4	216,513,210	305,060,750	3,009,856
16	Netherlands	7	1.4	291,308,450	489,421,170	1,403,296
17	India	6	1.2	113,495,990	141,752,655	996,144
18	United Arab Emirates	5	1	141,761,000	228,302,103	878,400
19	Singapore	5	1	38,287,830	62,498,340	315,424
20	Russia	5	1	68,979,000	93,778,830	657,104
21	Australia	4	0.8	55,227,310	73,106,095	496,032
22	Czechia	3	0.6	17,991,180	21,784,990	360,192
23	Israel	3	0.6	108,810,000	130,101,880	395,936
24	Switzerland	3	0.6	450,298,600	601,110,460	2,299,328
25	Spain	3	0.6	221,872,600	306,102,991	1,542,016
26	Finland	3	0.6	391,388,310	546,192,585	3,116,992
27	Turkey	2	0.4	15,097,610	22,373,010	82,048
28	Thailand	2	0.4	21,996,000	35,449,860	134,656
29	Slovenia	2	0.4	6,918,000	10,047,000	156,480
30	Ireland	2	0.4	6,697,330	13,160,450	152,320

Statistics from top500.org as of Nov-2025

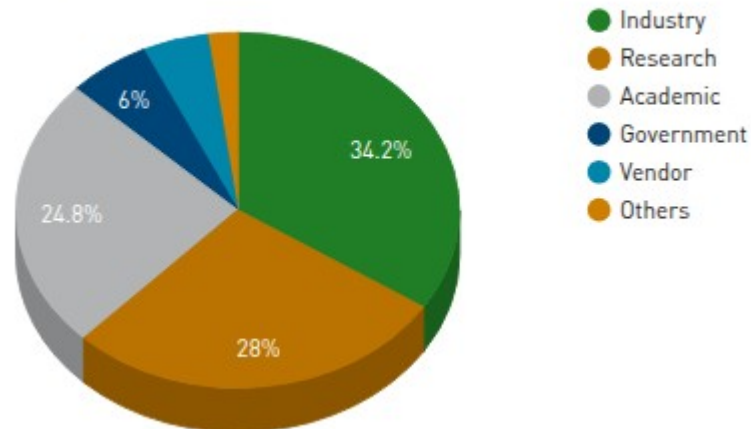
Vendors System Share



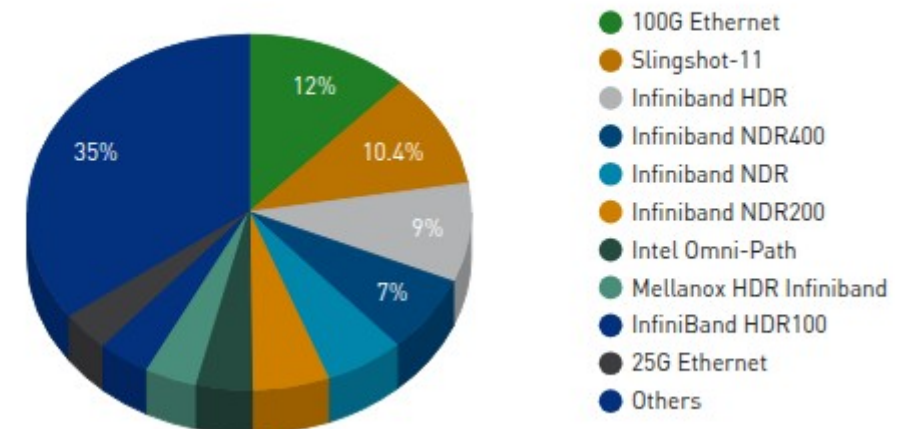
Accelerator/Co-Processor System Share



Segments System Share

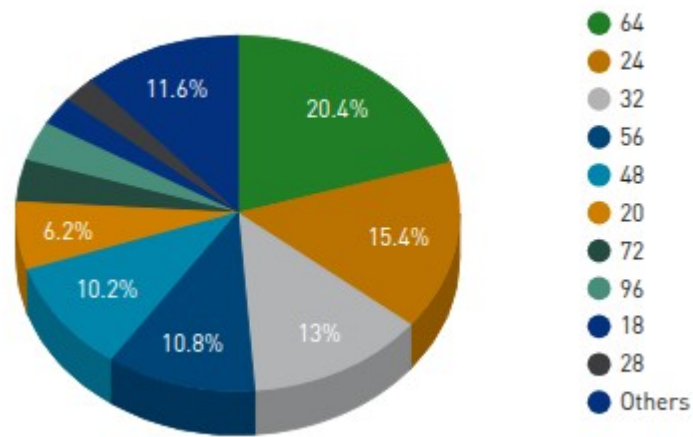


Interconnect System Share

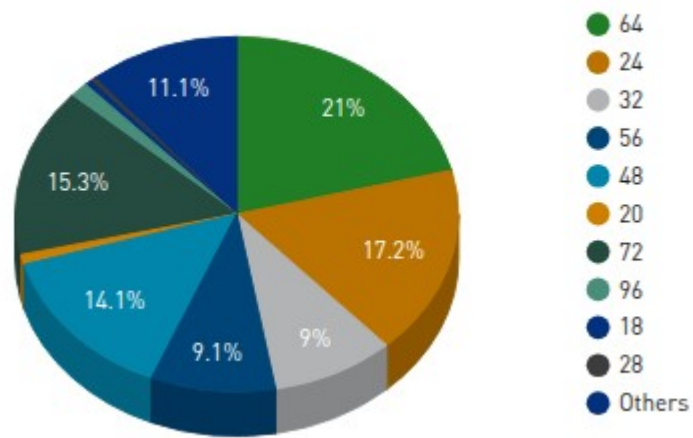


Cores per socket as of Nov-2025

Cores per Socket System Share



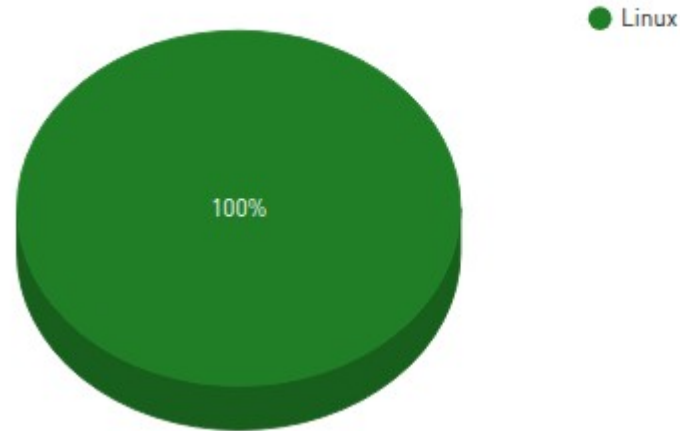
Cores per Socket Performance Share



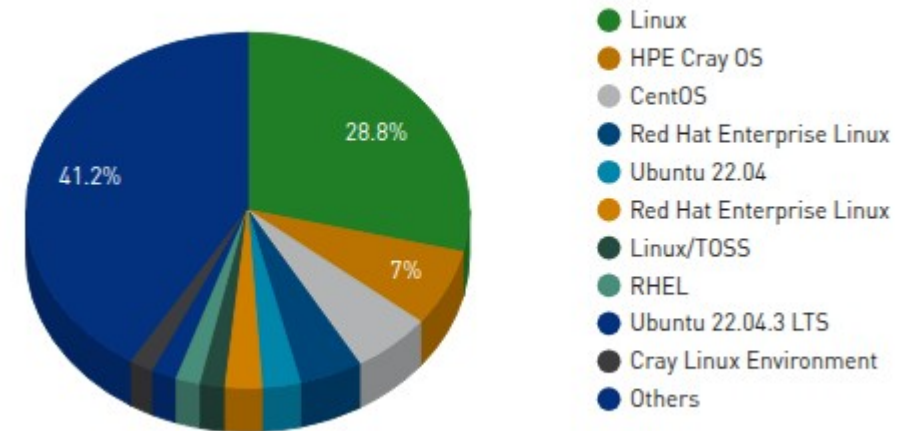
	Cores per Socket	Count	System Share (%)	Rmax (GFlops)	Rpeak (GFlops)	Cores
1	64	102	20.4	3,153,435,190	4,509,294,807	32,478,448
2	24	77	15.4	2,575,382,648	4,008,277,880	20,244,392
3	32	65	13	1,348,716,080	2,001,417,810	8,719,888
4	56	54	10.8	1,360,252,180	1,983,356,446	7,812,912
5	48	51	10.2	2,119,508,710	3,002,864,856	16,790,416
6	20	31	6.2	135,763,280	246,482,918	3,088,608
7	72	19	3.8	2,292,460,000	2,994,447,950	11,181,696
8	96	18	3.6	216,605,100	299,993,774	3,106,416
9	18	13	2.6	71,578,140	106,649,570	1,444,660
10	28	12	2.4	58,831,560	106,175,997	1,242,080
11	16	8	1.6	49,681,600	61,269,162	643,136
12	128	7	1.4	69,767,680	85,046,070	1,193,984
13	36	6	1.2	40,582,010	62,030,600	397,896
14	52	5	1	1,034,556,080	2,009,970,849	9,621,504
15	38	4	0.8	50,136,300	74,869,010	819,316
16	22	4	0.8	122,023,670	162,542,670	2,166,744
17	8	3	0.6	32,778,900	42,100,690	140,096
18	68	3	0.6	28,952,150	54,585,772	1,253,828
19	40	3	0.6	18,638,140	27,103,740	316,736
20	10	3	0.6	13,105,550	19,370,829	386,708
21	26	3	0.6	14,437,430	25,887,440	365,024
22	60	3	0.6	59,479,900	85,166,140	302,808
23	12	2	0.4	5,775,970	10,410,854	369,984
24	192	1	0.2	20,765,000	28,311,550	393,216
25	144	1	0.2	4,124,400	4,967,420	70,560
26	260	1	0.2	93,014,594	125,435,904	10,649,600
27	4	1	0.2	3,161,000	5,371,776	60,480

Statistics from top500.org as in Nov-2025

Operating system Family System Share



Operating System System Share



Programmers who use Linux in R&D units of Industry,
Academia and Government Labs

Linux is a must



- Dual boot
- Virtual Machine
- WSL2 (Windows Subsystem for Linux)
<https://learn.microsoft.com/en-us/windows/wsl/install>

Help available at my course:

<https://github.com/gphanikumar/mm2090>

Videos available as part of **System Commands** course of **Online B.Sc in Programming and Data Science**:

<https://github.com/gphanikumar/MM2090/blob/master/videos.md>

A typical cycle of code development

- Typical programming languages:
Fortran, C, C++ (including OpenCL, CUDA, SYCL ...)
- Profile code, identify hotspots
- Optimization of code / algorithm
- Choose a parallel paradigm
MPI / OpenMP / OpenACC / OpenCL / CUDA / ...
- Code, code properly, code efficiently ...
- Validate with serial version, bitwise
- Benchmark speedup
- Stop after reaching scalability limit



Home work

- How many cores are there in your m/c?
- What are the specs of those cores?
- How much RAM is consumed?
- Do you have any compilers installed?
- Determine how long your program runs

Profiling your code

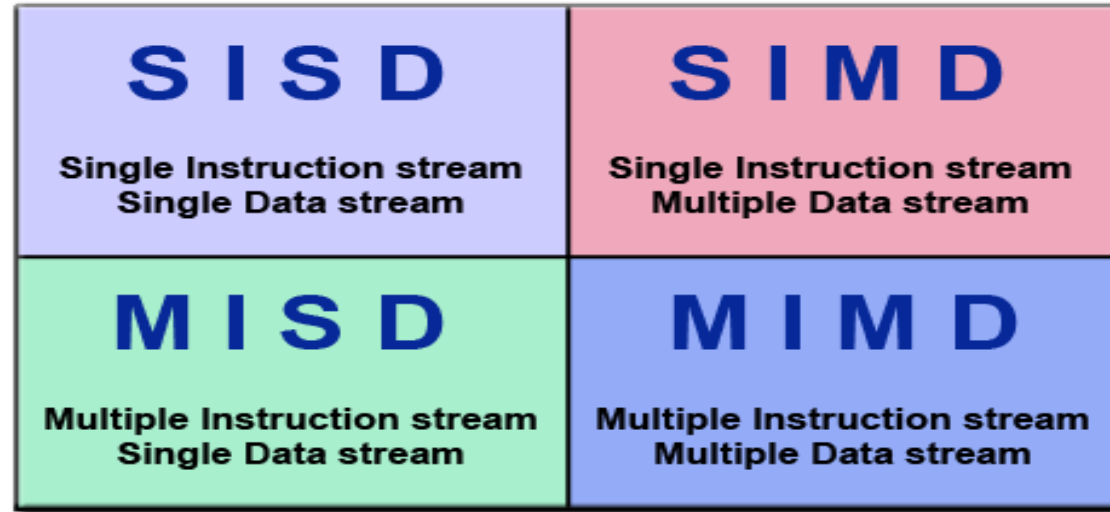
- Compile using `gcc` with “`-pg`” option to enable profiling
- Execute a representative run to generate “`gmon.out`”
- Run `gprof` to generate profile statistics of your code
- Identify the routine that hogs most of cpu and parallelize that first

Git clone this repo:

`https://github.com/gphanikumar/hpc`

Some HPC terminology

Flynn's Taxonomy

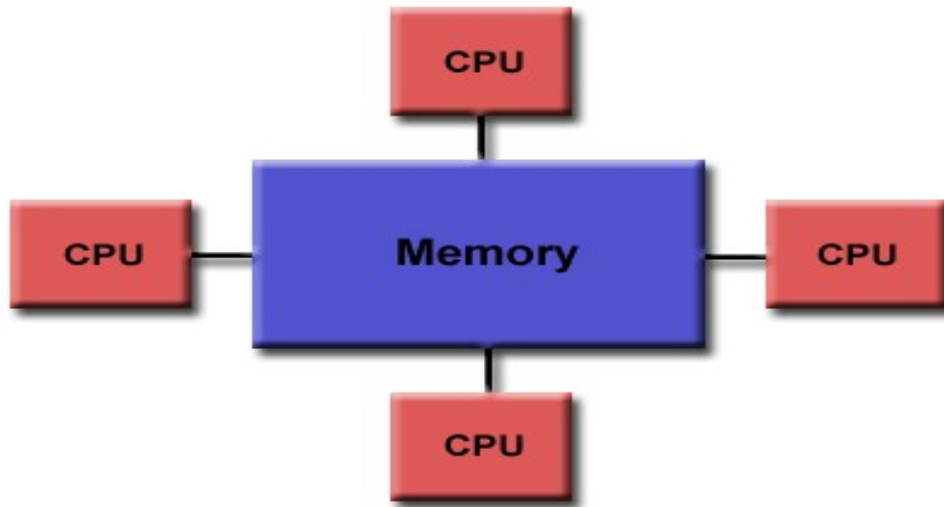


Some more keywords:

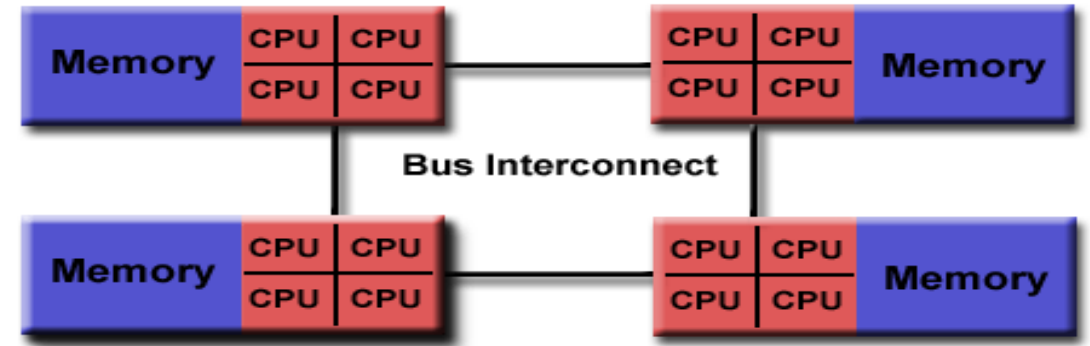
Node
CPU / Socket / Processor / Core
Task
Pipelining
Shared Memory
Distributed Memory
Symmetric Multiprocessor (SMP)
Communications
Synchronization

Latency
Bandwidth
Granularity
Observed speed-up
Parallel overhead
Massively Parallel (MP)
Embarrassingly Parallel (EP)
Scalability

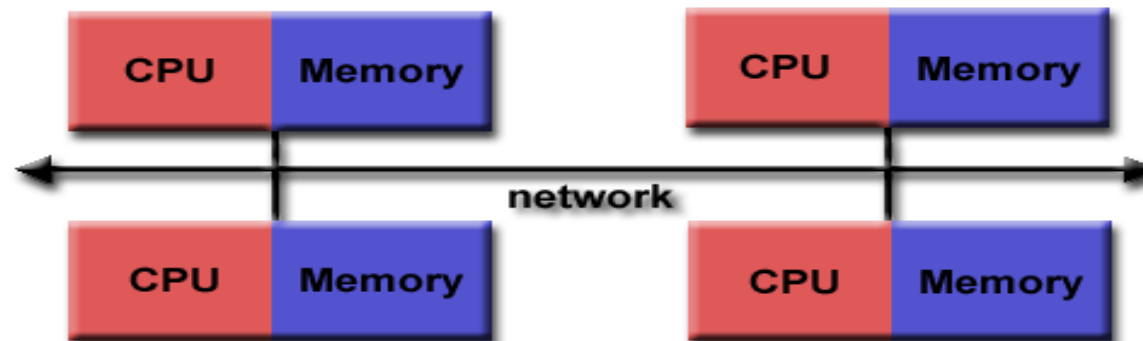
Memory Architectures



Shared Memory : Uniform Memory Access

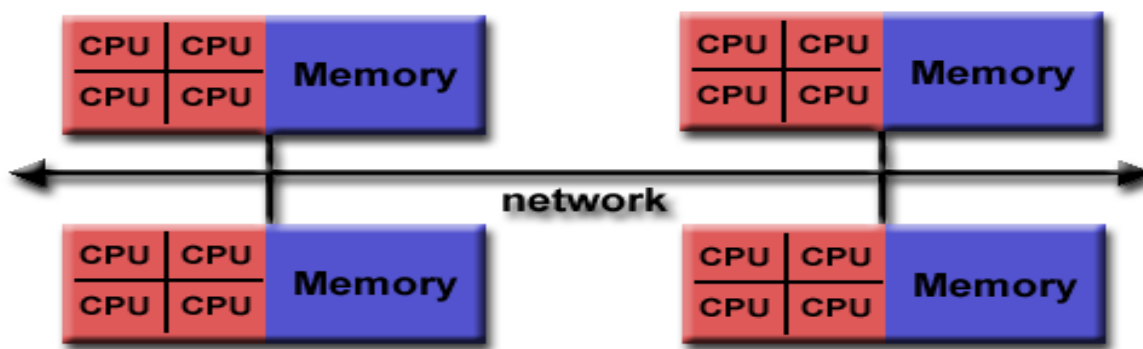


Shared Memory : Non-uniform Memory Access

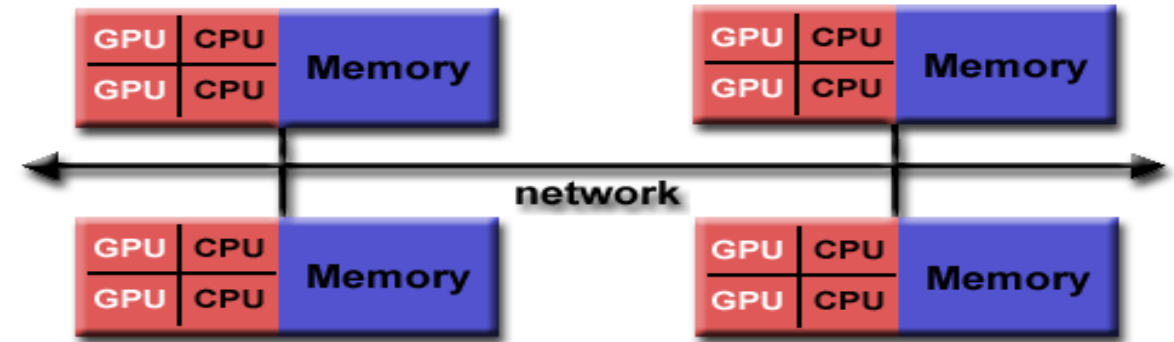


Distributed Memory

Hybrid Architectures



Both shared and distributed memory architectures :
only CPUs

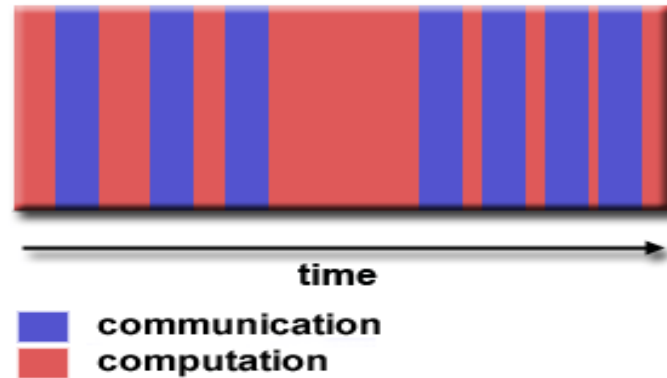


Both shared and distributed memory architectures : CPU
+ GPU

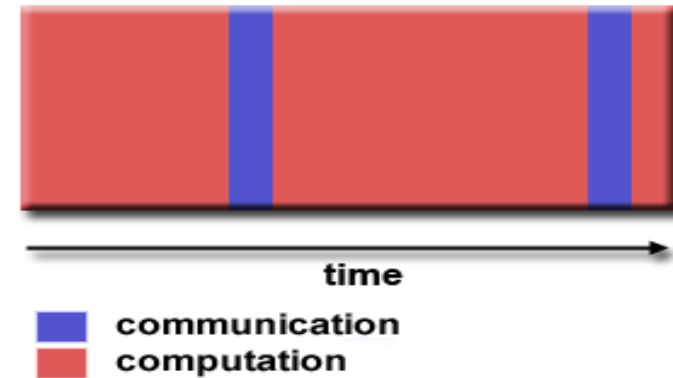
Watchout: Latency and bandwidth for data transfer between different memory locations

Granularity

Fine grained parallelism

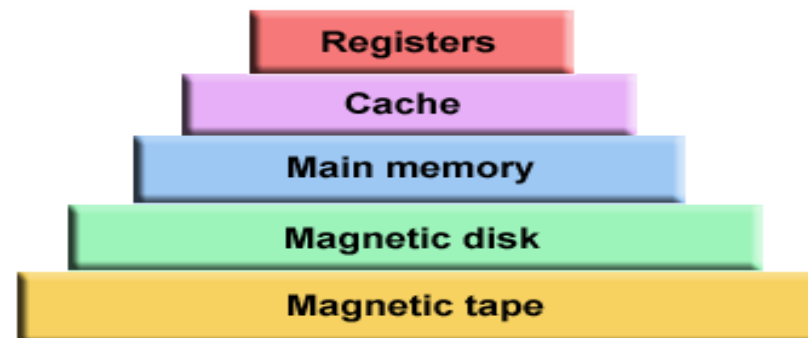


Coarse grained parallelism



I/O could be expensive

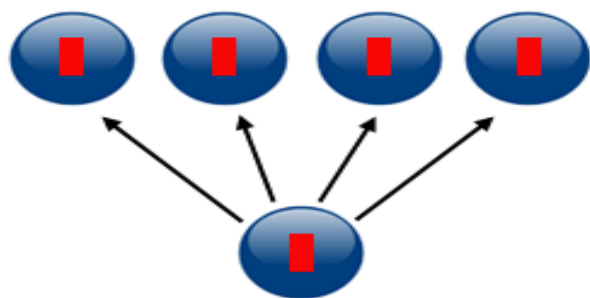
Memory Hierarchy



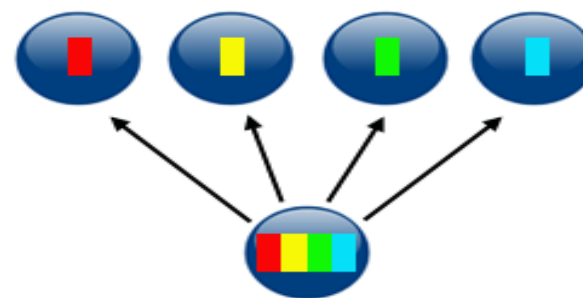
1 ns	1x
10 ns	10x
100 ns	100x
100 ms	100,000,000x
10 s	1e+10x

- Latency
- Bandwidth

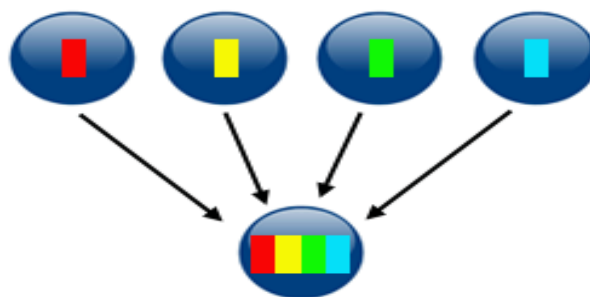
Scope of Communications



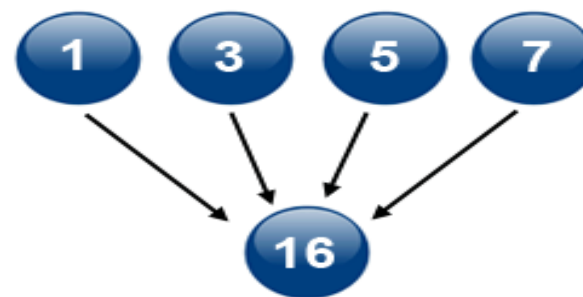
broadcast



scatter

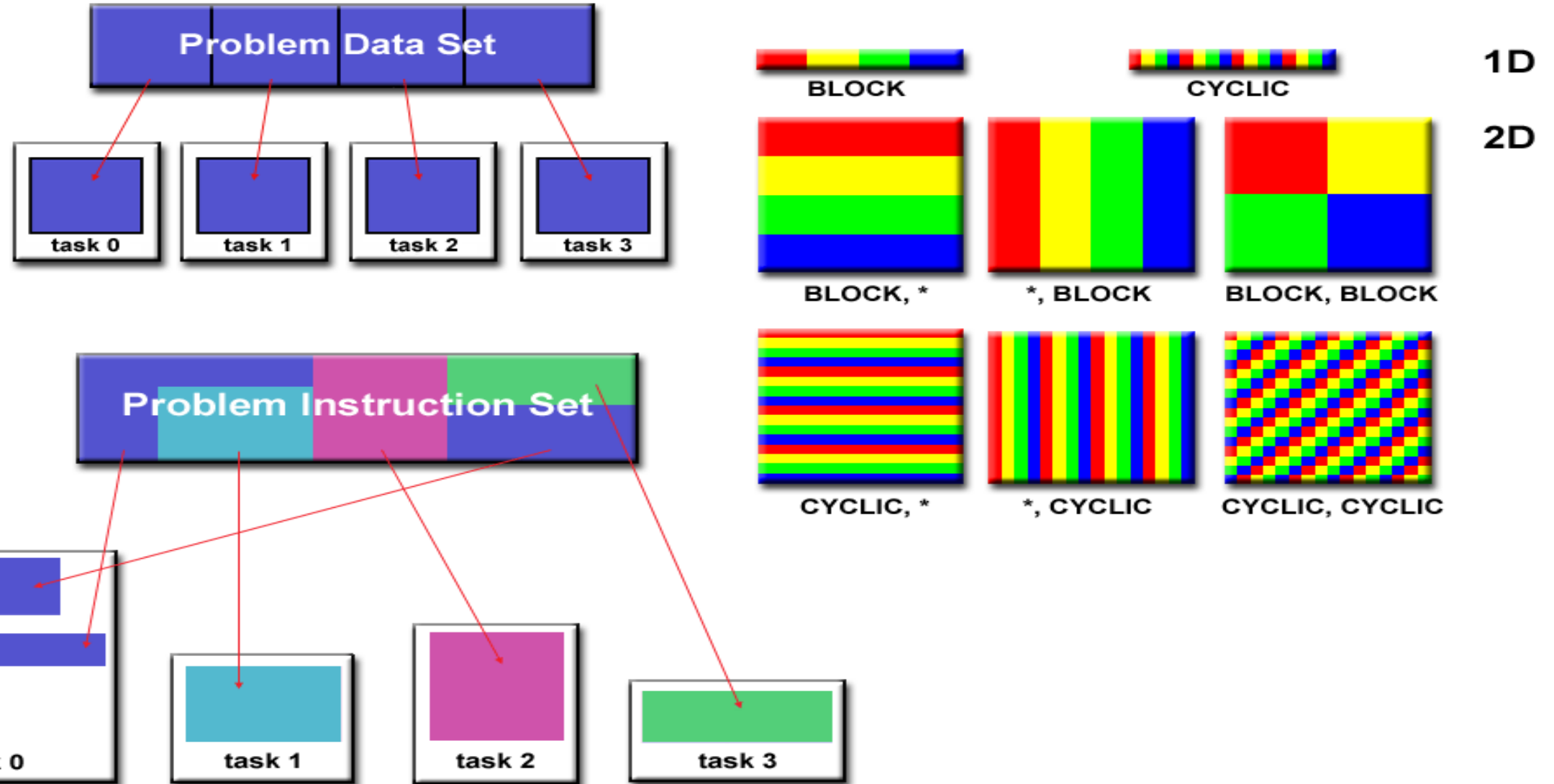


gather



reduction

Domain Decomposition



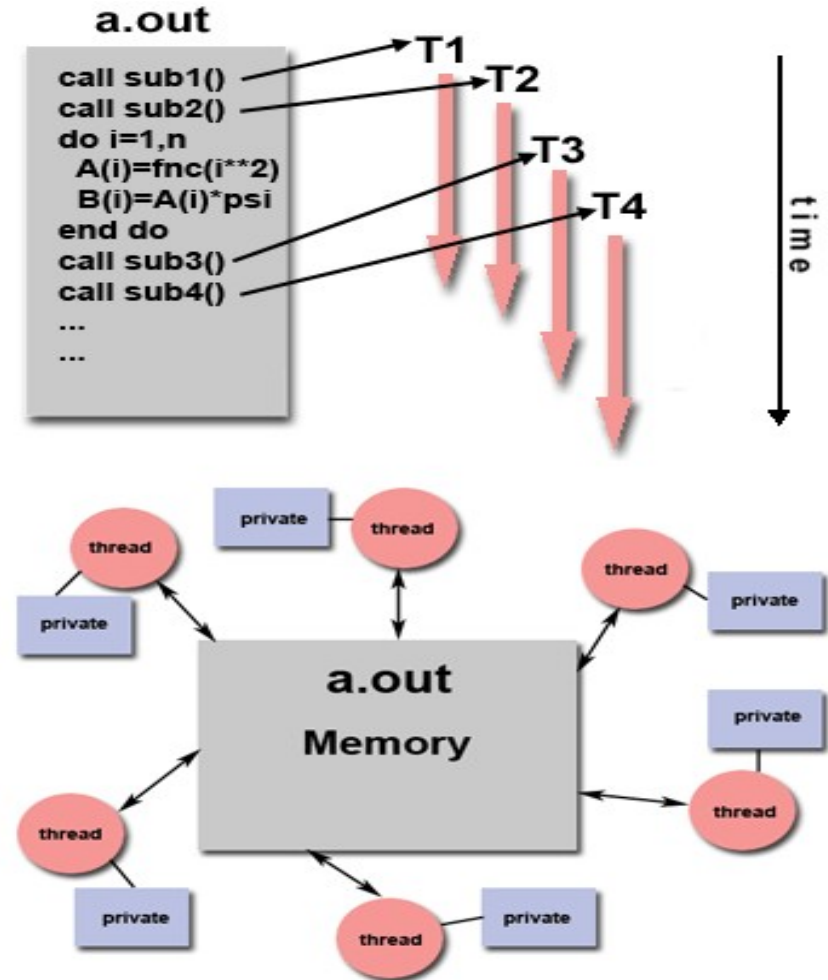
Parallel Processing : Threads Model

POSIX Threads

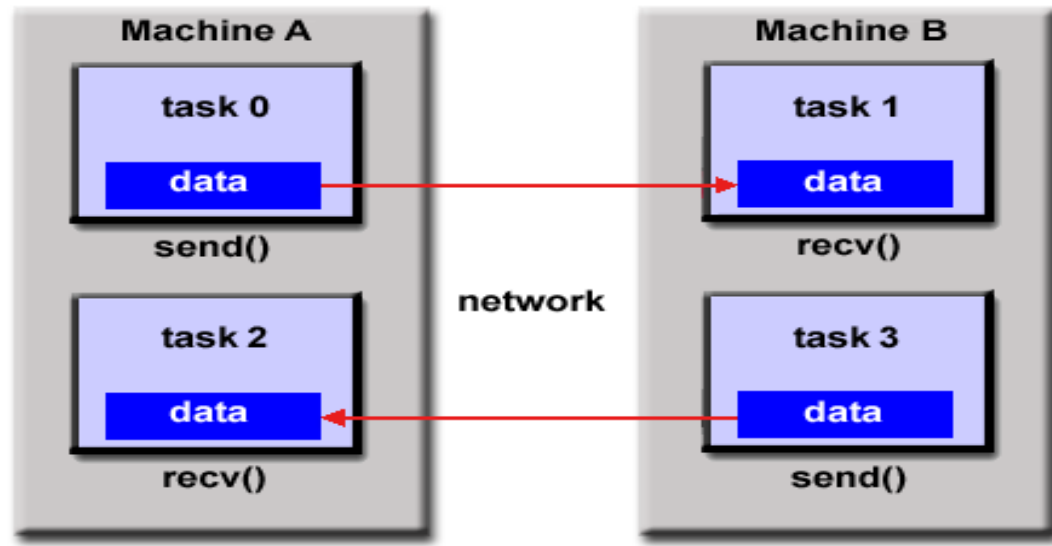
Specified by the IEEE POSIX 1003.1c standard (1995). C Language only.
Part of Unix/Linux operating systems
Library based
Commonly referred to as Pthreads.
Very explicit parallelism; requires significant programmer attention to detail.

OpenMP

Industry standard, jointly defined and endorsed by a group of major computer hardware and software vendors, organizations and individuals.
Compiler directive based
Portable / multi-platform, including Unix and Windows platforms
Available in C/C++ and Fortran implementations
Can be very easy and simple to use - provides for "incremental parallelism". [Can begin with serial code.](#)



Parallel Processing : Message Passing Model



A set of tasks that use their own local memory during computation. Multiple tasks can reside on the same physical machine and/or across an arbitrary number of machines.

Tasks exchange data through communications by sending and receiving messages.

Data transfer usually requires cooperative operations to be performed by each process. For example, a send operation must have a matching receive operation.

MPI : Message Passing Interface

<https://www.open-mpi.org/>

Profiling and Debugging



Intel Inspector



LIVERMORE COMPUTING CENTER
HIGH PERFORMANCE COMPUTING

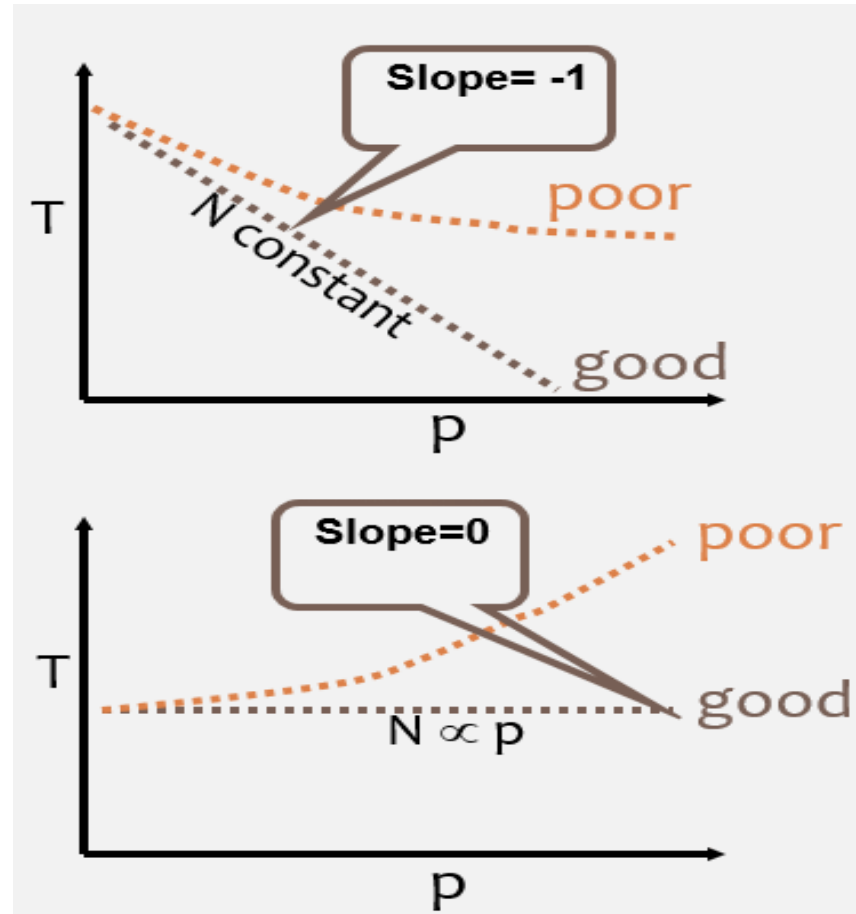
Scaling

- Strong scaling

The total problem size stays fixed as more processors are added.

Goal is to run the same problem size faster

Perfect scaling means problem is solved in $1/P$ time (compared to serial)



- Weak scaling

The problem size per processor stays fixed as more processors are added. The total problem size is proportional to the number of processors used.

Goal is to run larger problem in same amount of time

Perfect scaling means problem $P \times$ runs in same time as single processor run

Limits and Cost of Parallel Programming

- Amdahl's law
- Complexity : Design, Coding, Debugging, Tuning, Maintenance
- Portability : Vendor enhancements, non-standard APIs or libraries
- Resource Requirement : CPU time, Memory

$$\text{Speed up} = 1/(1-F)$$

F is fraction of code parallelized

Batch mode execution

- A script is used to instruct job scheduler
- Execution is in a separate environment than head node
- All resources required for the task to be assessed upfront
- Programs that can stop and continue are preferable
- Do not assume availability of direct internet access from the compute nodes
- Remember: often, a Linux terminal with a shell is all you have to interact with the HPC

You need to know shell scripting

Job Schedulers

LoadLeveler	IBM	Renamed to Tivoli Workload Scheduler LoadLeveler
Terascale open-source resource and queue manager (TORQUE)	Adaptive Computing	Since 2003
Open PBS (Portable Batch System)	Free and opensource	Since 1998
PBS Pro	Opensourced by Altair	Part of Open HPC project
Sun Grid Enginer (SGE), Computing in Distributed Networked Environment (CODINE), Global Resource Director (GRD)	Oracle Grid Engine	Since 2000 under Sun. Predates AWS on a public cloud.
Simple Linux Utility for Resource Management (SLURM)	GNU Licensed	Most popular on top500
Platform load sharing facility (LSF)	IBM Spectrum LSF Suite	Origins in University of Toronto under Utopia project → OpenLava → Platform Computing
BProc + Maui	Beowulf Clustering Project	Since 1994 at NASA to help build clusters out of regular desktop machines.

File i/o in parallel computing

- Reading from & Writing to files is expensive
- Note the delay between program file i/o and actual disc i/o
- Separate files for each compute node could be preferable
- Consider self describing file formats such as HDF5
- Use scripts to merge files before taking them out of HPC environment
- Compress using tools & options considering time to extract vs. file size
- Consider block size in the file system to avoid too many tiny files
- Number of files & depth of directory structure could be a bottleneck
- Do not assume you can take all your data out for analysis

Data Visualization in HPC

- Large programs lead to huge data
- Resources on Workstation used for visualization vs. execution
- Scripts to generate views vs. live exploration
- Consider scripts to “[bin](#)” the data for quick visualization
- Data as time series of files
- Data as separate files from each compute node
- Consider [paraview](#) that understands parallel computing

$$\frac{1}{m^n}$$

Version Control

Use git for source code maintenance

Version number convention

Data type for numbers

Arbitrary precision arithmetic

Double vs. Float vs. Half-Float

Integer computation

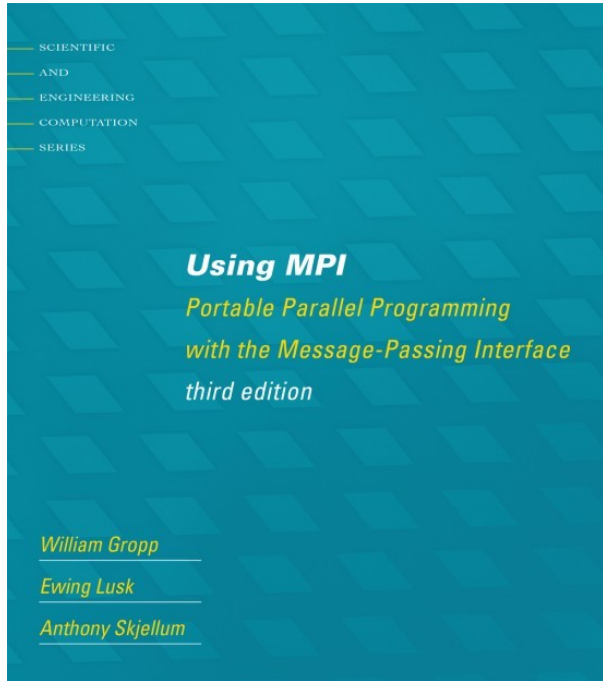
Speed for each type is not the same !

Computing skills recommended

- Terminal : bash, ssh, scp, rsync, wget, ...
- Text editor: vi, emacs, nano, ...
- Network : VPN, unconventional ports, file transfer hops, ...
- Authentication : ssh keys, authentication tokens, ...
- Licenses: node locked licenses, site licenses, usb dongles, internet authenticated subscriptions, ...

Free and opensource programs are recommended

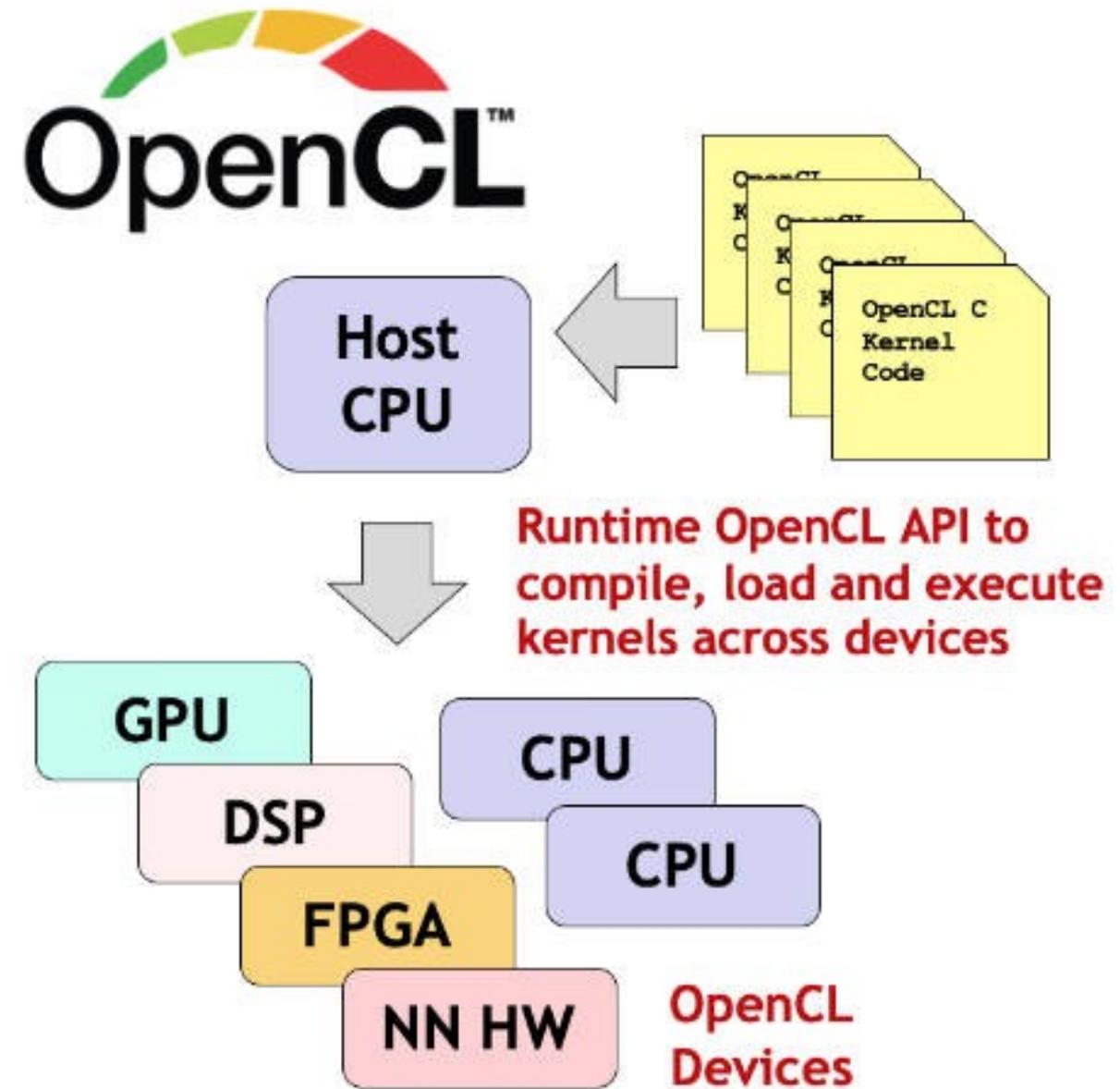
HPC Paradigms



OpenMP®



- OpenCL™ (Open Computing Language) is an open, royalty-free standard for cross-platform, parallel programming of diverse accelerators found in supercomputers, cloud servers, personal computers, mobile devices and embedded platforms.
- Low-level Parallel Programming
- Standards maintained by Khronos group

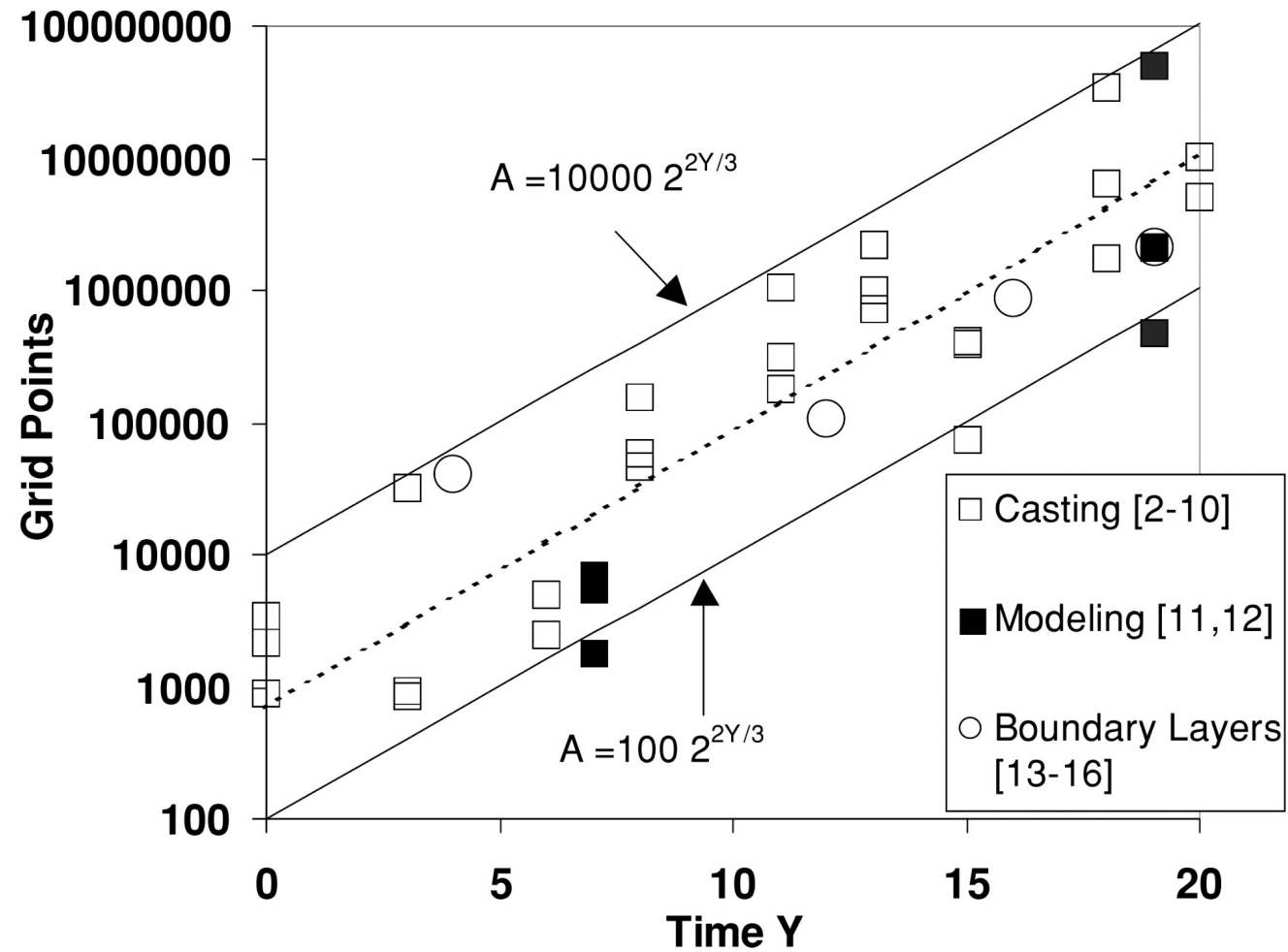


- Higher-level programming model
- C++ abstraction layer for OpenCL
- Single source programming style
- SYCL makes the development easier
 - May effect global performance



HPC applications

Moore's Law and Numerical Modeling



HPC in materials science

- ACM Gordon Bell Prize 2011
- Peta-scale Phase-Field Simulation for Dendritic Solidification on the TSUBAME 2.0 Supercomputer

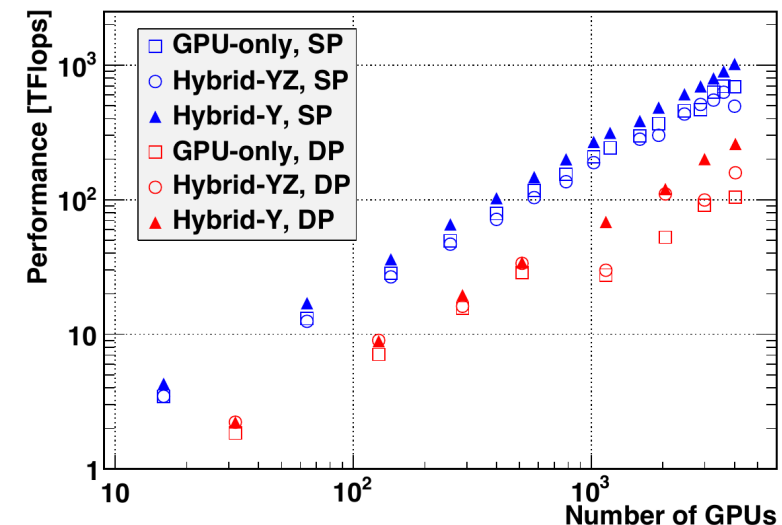
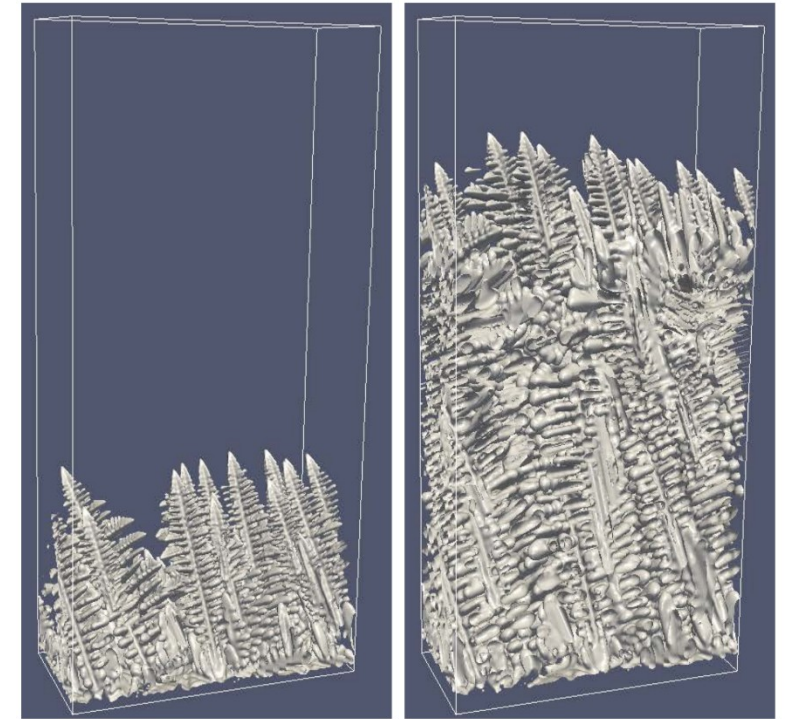
4032 GPUs

16128 CPUs

$4096 \times 6500 \times 10400$ grid points

Shimokawabe *et.al*, ACM v3 p11 (SC 11 – 2011)
https://awards.acm.org/award_winners

2023 winners include a team from IISc and Vishal Subramanian (DD alumnus from our group)



The enormity of microstructure simulation

Grid Size : 10 nm

Time Step: 1 ns

Number of Grid Points: 2000 x 2000 x 2000

Time Steps: 1,000,000

Number of variables: 5+

Established, optimized, generic code ?

Microstructure Simulation is a HPC problem !!

MicroSim

- Uses **MPI**, **OpenCL** and **CUDA**(Nvidia) for parallelizing the solvers over multicores of CPU and GPU.
- Utilizes all the computing resources available and thus can be run in **Workstations and Clusters** by making minor changes in the Infiles.

- Include Multiple Solvers capable of rendering **Multiphase** and **Multicomponent** visualizations in 2D as well as in 3D

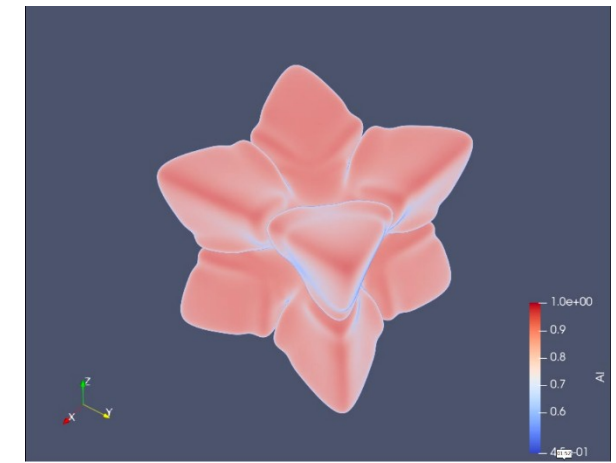
- Includes a parser for **extracting thermodynamic data** from both encrypted and unencrypted calphad **thermodynamic databases**.

- Has a simple and intuitive **GUI** that helps to import/edit/create infiles for running the solvers.
- Also bundled with a post processing tool that can visualize, export etc. without leaving the app.

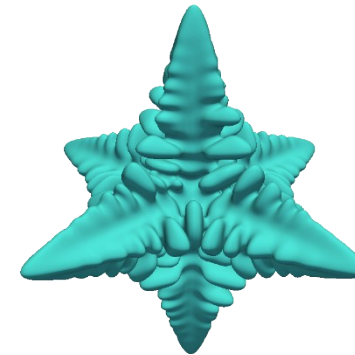
<https://github.com/ICME-India/MicroSim>



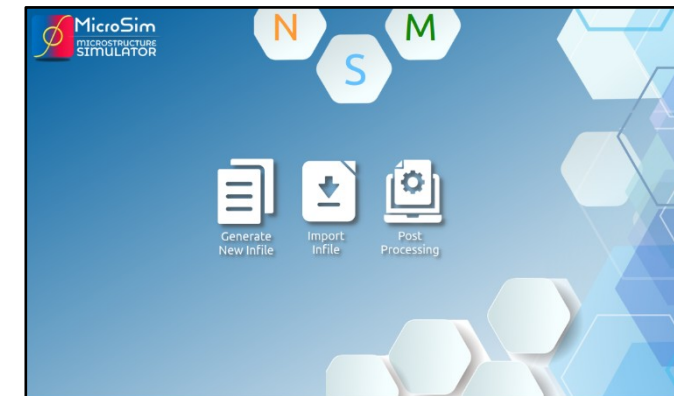
Grain orientation evolution



' precipitate

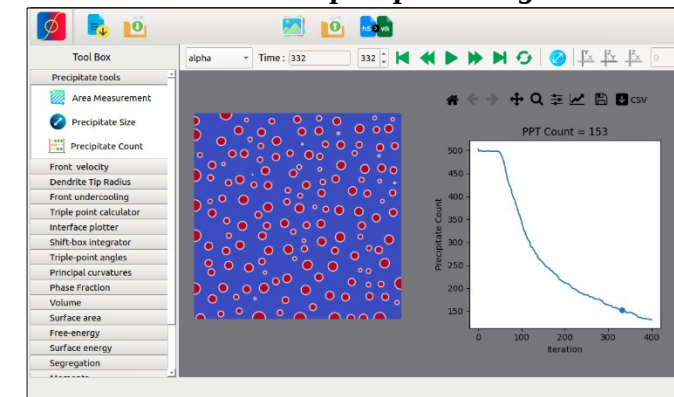


Dendrite Evolution



GUI and post-processing tools

Developed By:



<https://github.com/ICME-India/MicroSim>



@IISc: Abhik Choudhury & group



@IIT Bombay: M.P. Gururajan & group



@IIT Hyderabad: Saswata Bhattacharyya & group



@IIT Madras: Gandham Phanikumar & group



@S.P.University Pune: Vaishali Shah & group



@CDAC Pune: Venkatesh Shenoy & group



Contents lists available at ScienceDirect

Computational Materials Science

journal homepage: www.elsevier.com/locate/commatsci



Full length article

MicroSim: A high-performance phase-field solver based on CPU and GPU implementations

Tanmay Dutta^a, Dasari Mohan^{b,e}, Saurav Shenoy^c, Nasir Attar^d, Abhishek Kalokhe^d,
Ajay Sagar^a, Swapnil Bhure^a, Swaroop S. Pradhan^b, Jitendriya Praharaj^a, Subham Mridha^a,
Anshika Kushwaha^a, Vaishali Shah^f, M.P. Gururajan^e, V. Venkatesh Shenoi^d,
Gandham Phanikumar^b, Saswata Bhattacharyya^c, Abhik Choudhury^{a,*}

^a Department of Materials Engineering, Indian Institute of Science, Bangalore, 560012, Karnataka, India

^b Department of Metallurgical and Materials Engineering, IIT Madras, Chennai, 600036, Tamil Nadu, India

^c Department of Materials Science and Metallurgical Engineering, IIT Hyderabad, Hyderabad, 502284, Telangana, India

^d Center for development of Advanced Computing, Pune University Campus, Pune, 411007, Maharashtra, India

^e Department of Metallurgical Engineering and Materials Science, IIT Bombay, Powai, Mumbai, 400076, Maharashtra, India

^f Department of Scientific Computing, Modeling and Simulation, Savitribai Phule Pune University, Pune, 411007, Maharashtra, India



Microstructure evolution as a HPC case study

Using

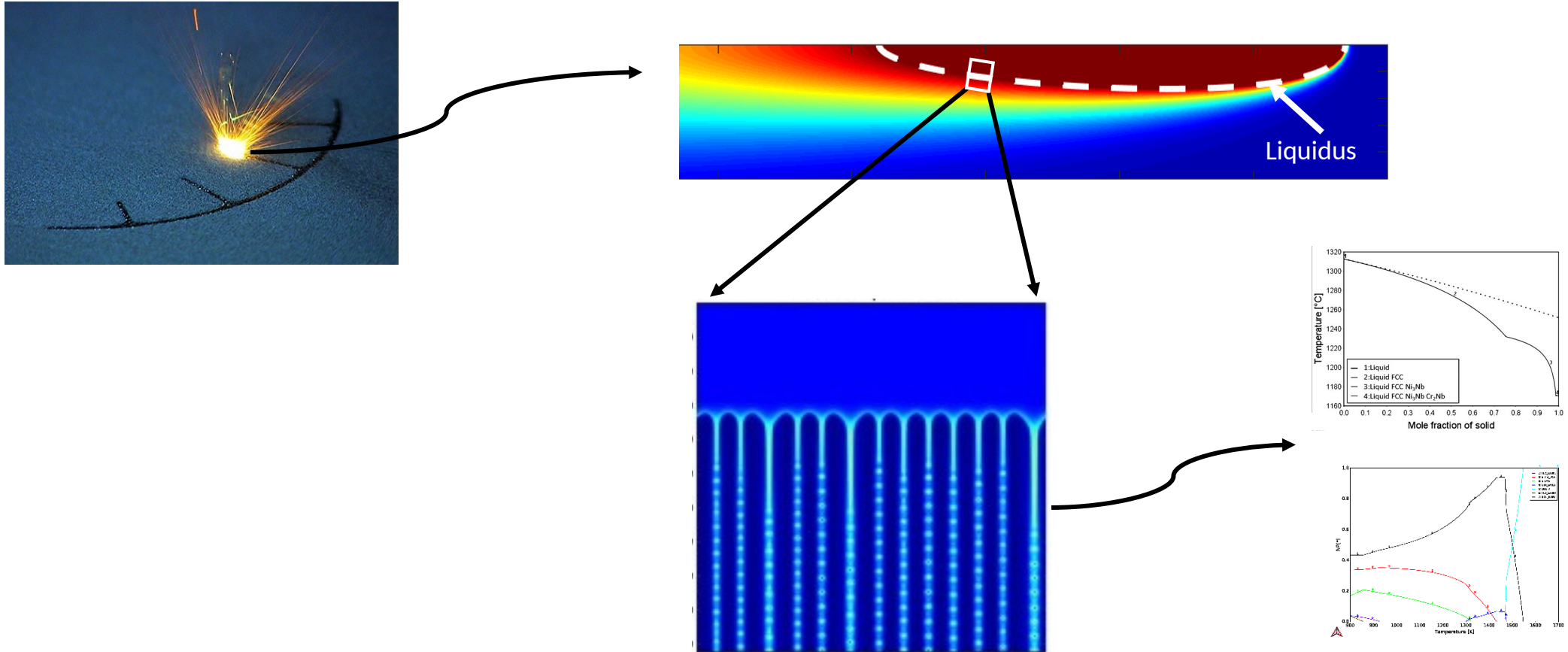
MicroSim: A suite of multi-CPU, multi-GPU PDE solvers

<https://github.com/ICME-India/microsim>

<https://microsim.co.in/>

Additive Manufacturing

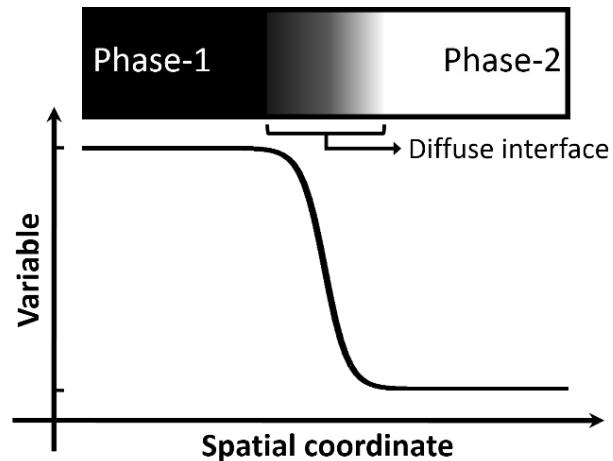
- Multiscale in additive manufacturing



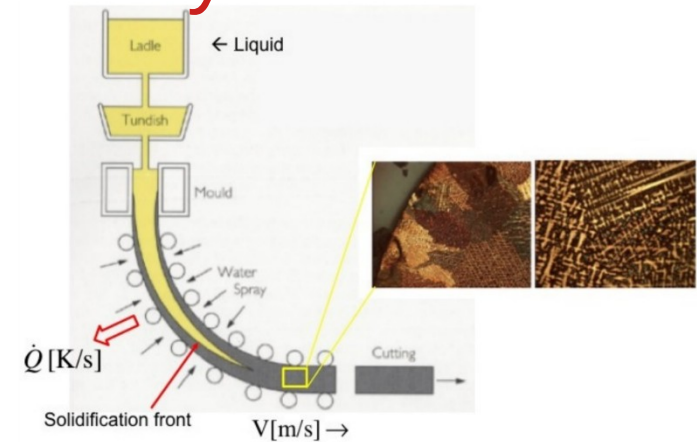
Schematic of multiscales involved in additive manufacturing

Microstructure Evolution as a HPC case study

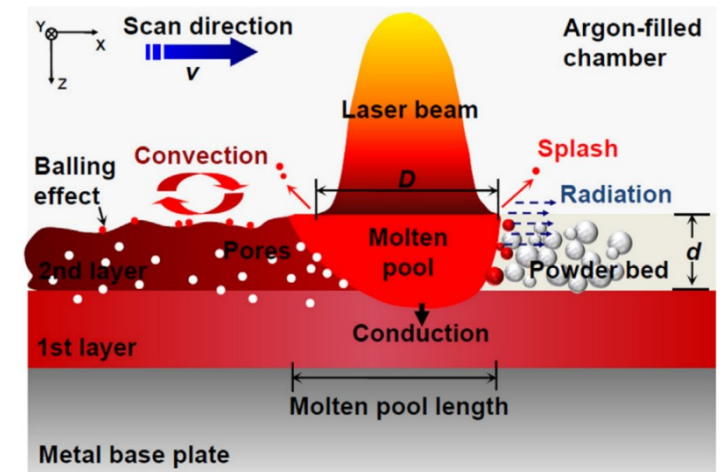
- Manufacturing process involves in solidification
 - Continuous casting
 - Welding
 - Additive manufacturing
- Phase-field model for solidification simulation
 - $\phi = 1$: Solid
 - $\phi = 0$: Liquid
 - $0 < \phi < 1$: Interface



Schematic of diffuse interface



Continuous casting



Laser additive manufacturing

Phase-field model

- Phase-field evolution

$$\frac{1}{M_\phi} \frac{\partial \phi}{\partial t} = \varepsilon_\phi^2 \nabla^2 \phi - W \frac{\partial p(\phi)}{\partial \phi} - \frac{\partial h_p(\phi)}{\partial \phi} \left[f^S - f^L - \sum_{i=1}^n (c_i^S - c_i^L) \tilde{\mu}_i \right]$$

- Composition evolution

$$\frac{\partial c_i}{\partial t} = \nabla \cdot [1 - h_d(\phi)] \sum_{j=1}^n D_{ij}^L \nabla c_j^L + \nabla \cdot j_{at}$$

$$j_{at} = \frac{\varepsilon_\phi}{\sqrt{2W}} (c_i^L - c_i^S) \frac{\partial \phi}{\partial t} \frac{\nabla \phi}{|\nabla \phi|}$$

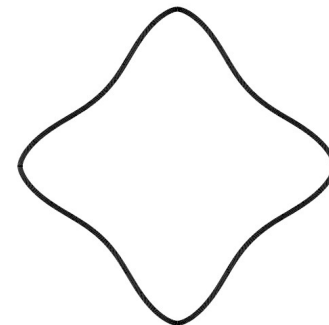
- Thermodynamic input: Gibbs energy of each phase

- Anisotropy

$$\varepsilon(\theta) = \varepsilon_0 (1 + \epsilon_4 \cos(4\theta))$$

- Directional solidification

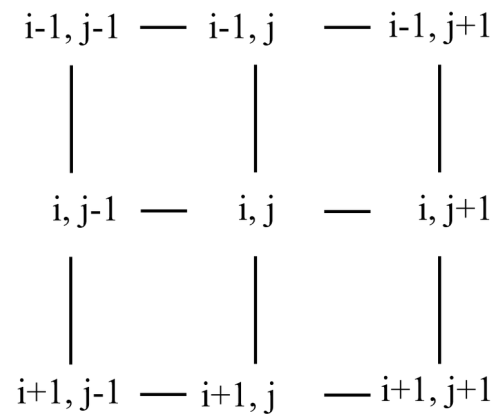
$$T(x) = T_0 + T_G(x - V_p t)$$



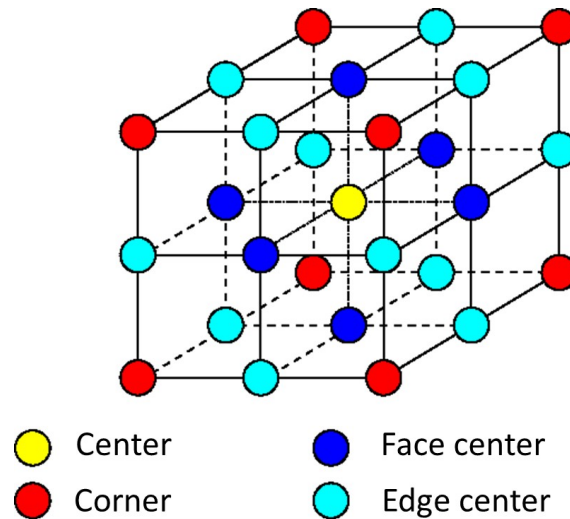
4-fold anisotropy

Implementation

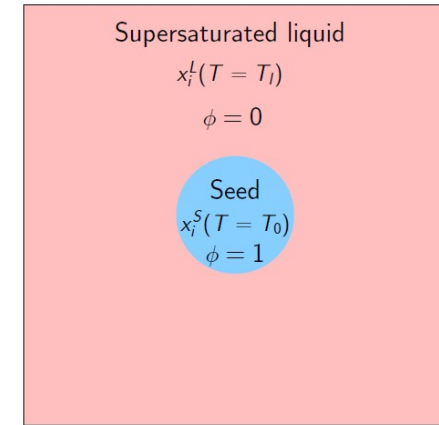
- Finite Difference Method - Euler Explicit method for time and space
- Central difference method for gradient
- Stencils



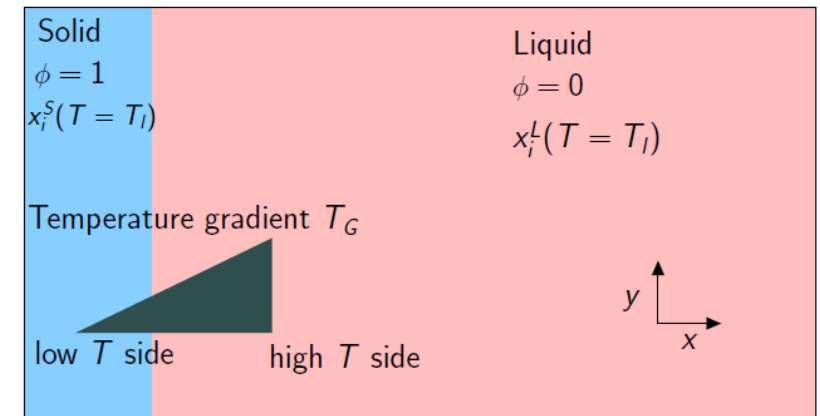
9 - point stencil



27 - point stencil



Schematic of isothermal domain

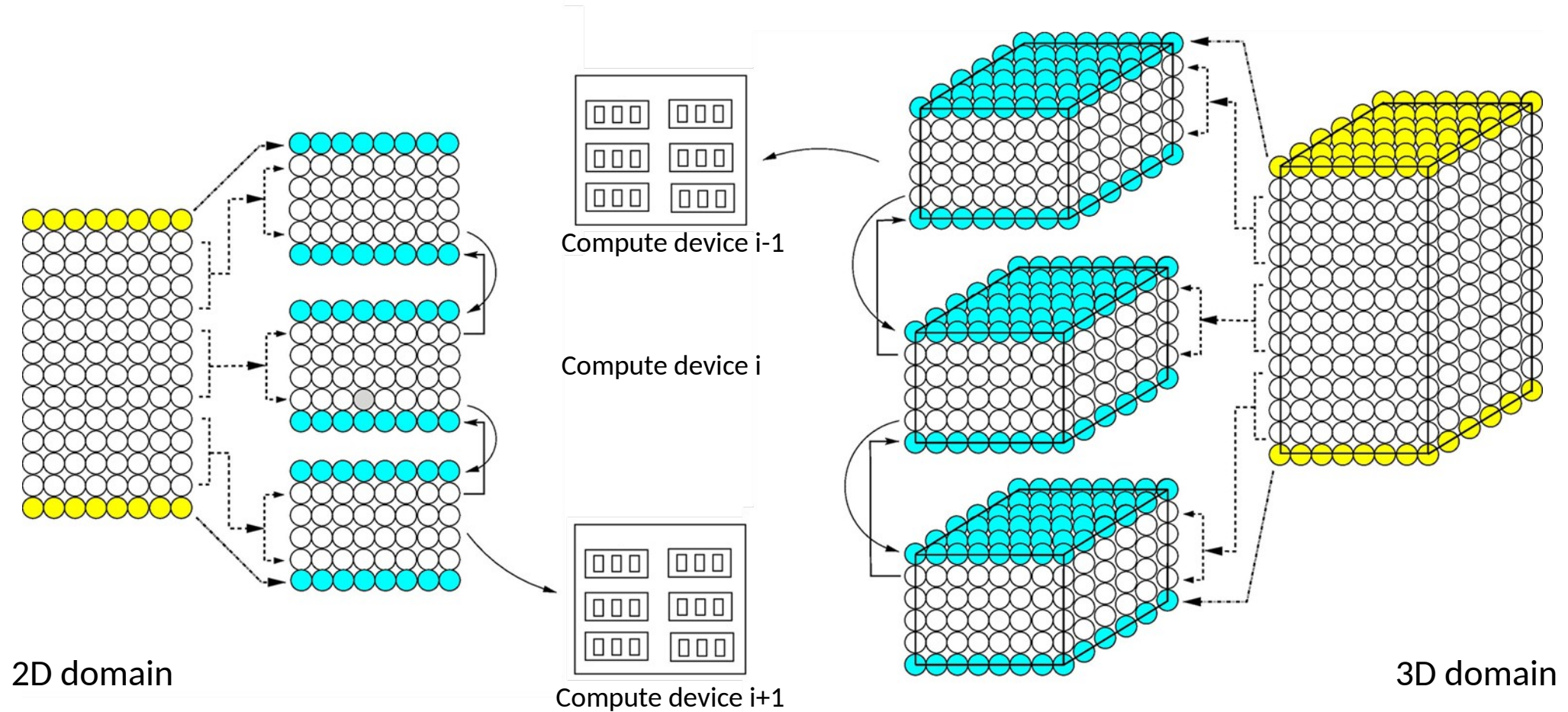


Schematic of directional solidification domain

- Phase-field simulations are computationally expensive

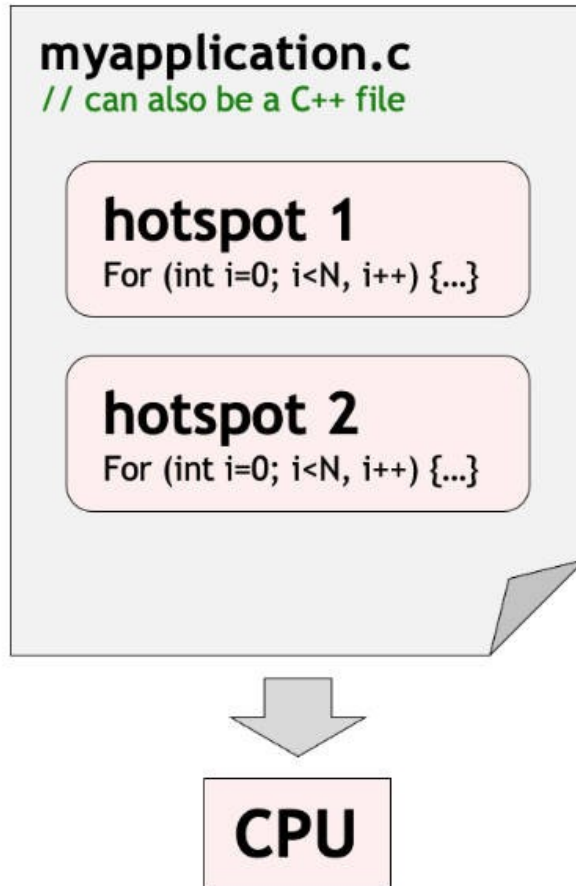
Implementation

- Domain decomposition for Message Passing Interface (MPI)

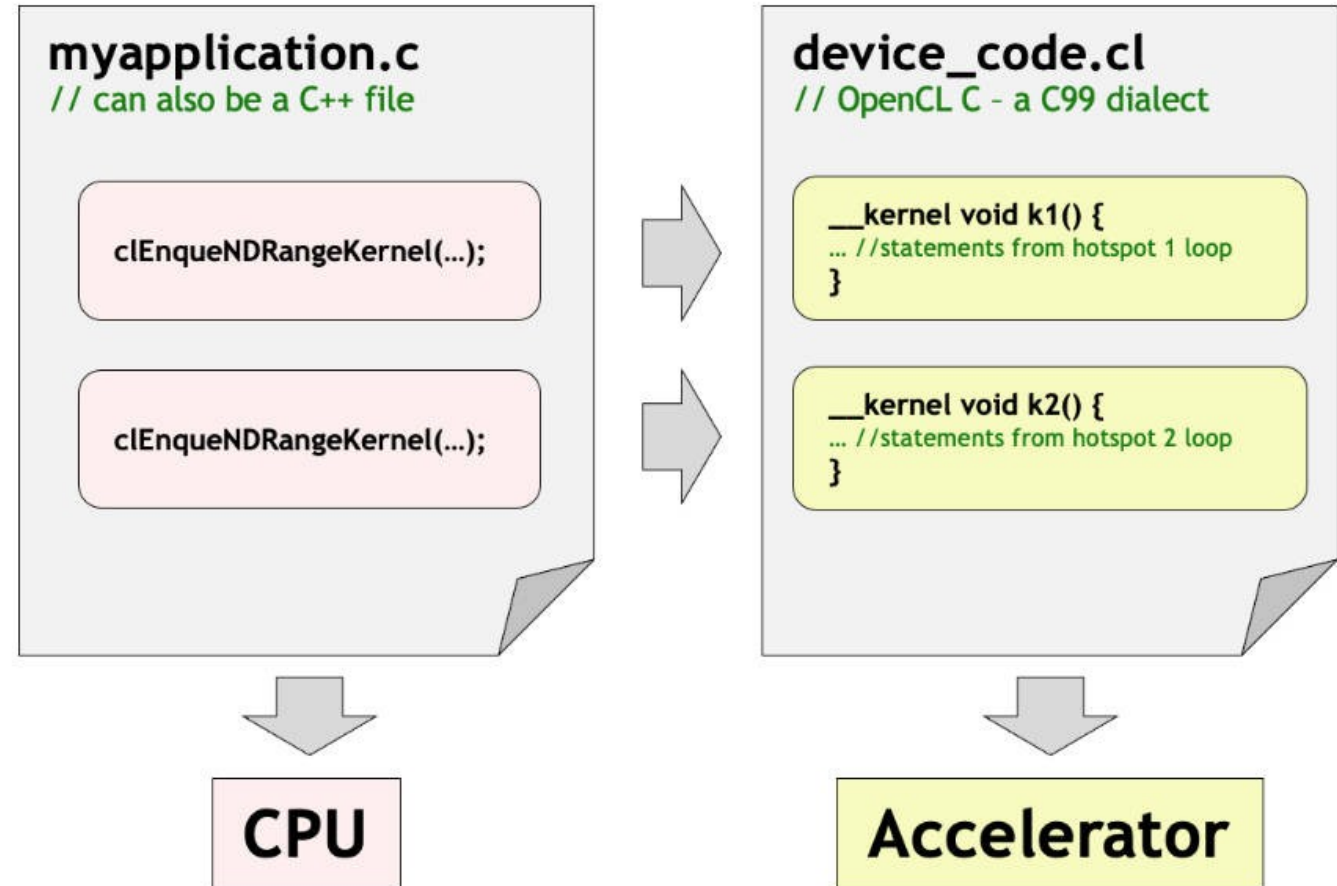


OpenCL Programming Model

C/C++ Programming

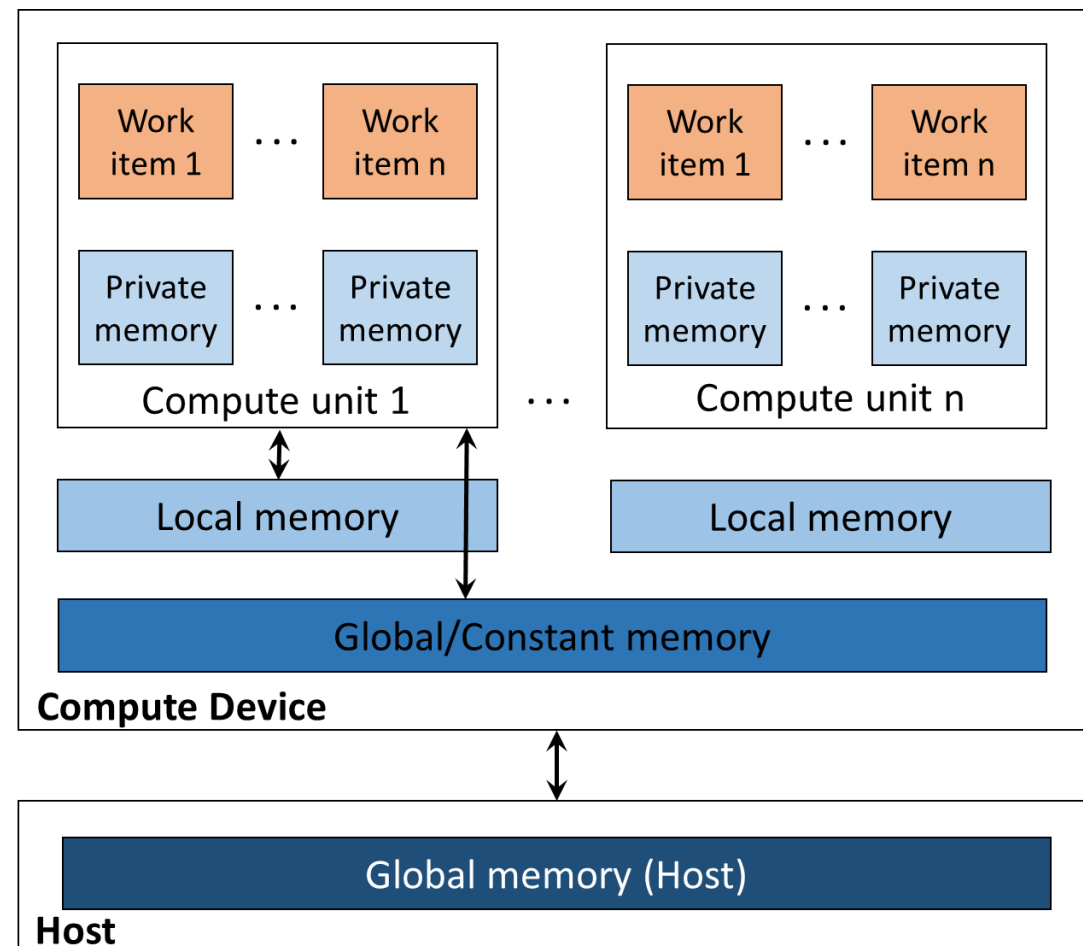


OpenCL Programming



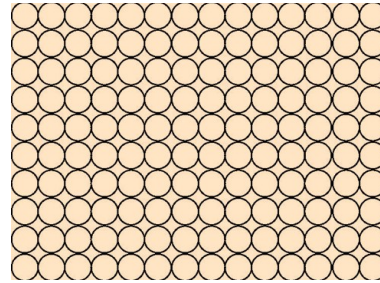
GPU implementation

- OpenCL 1.2: Latest OpenCL 3.0
 - AMD® CPUs/GPUs
 - NVIDIA® GPUs
 - Intel® CPUs/GPUs
- Host code
 - Device query
 - Memory allocation
 - Context creation
 - Command queues
- Device code
 - Kernels



OpenCL – Memory hierarchy model

GPU implementation



Computational Domain

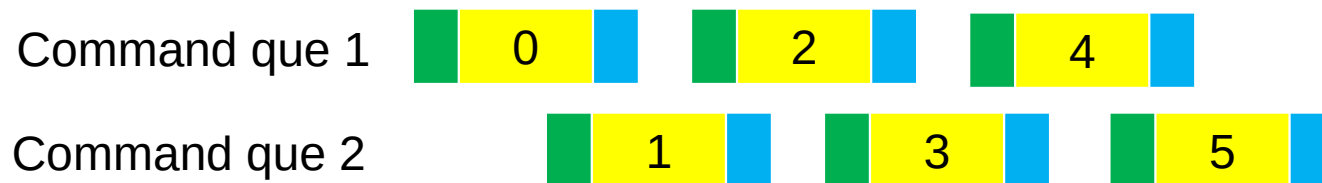
Standard GPU Computation: Memory limitation



Serial data transfer and computation: Performance limitation



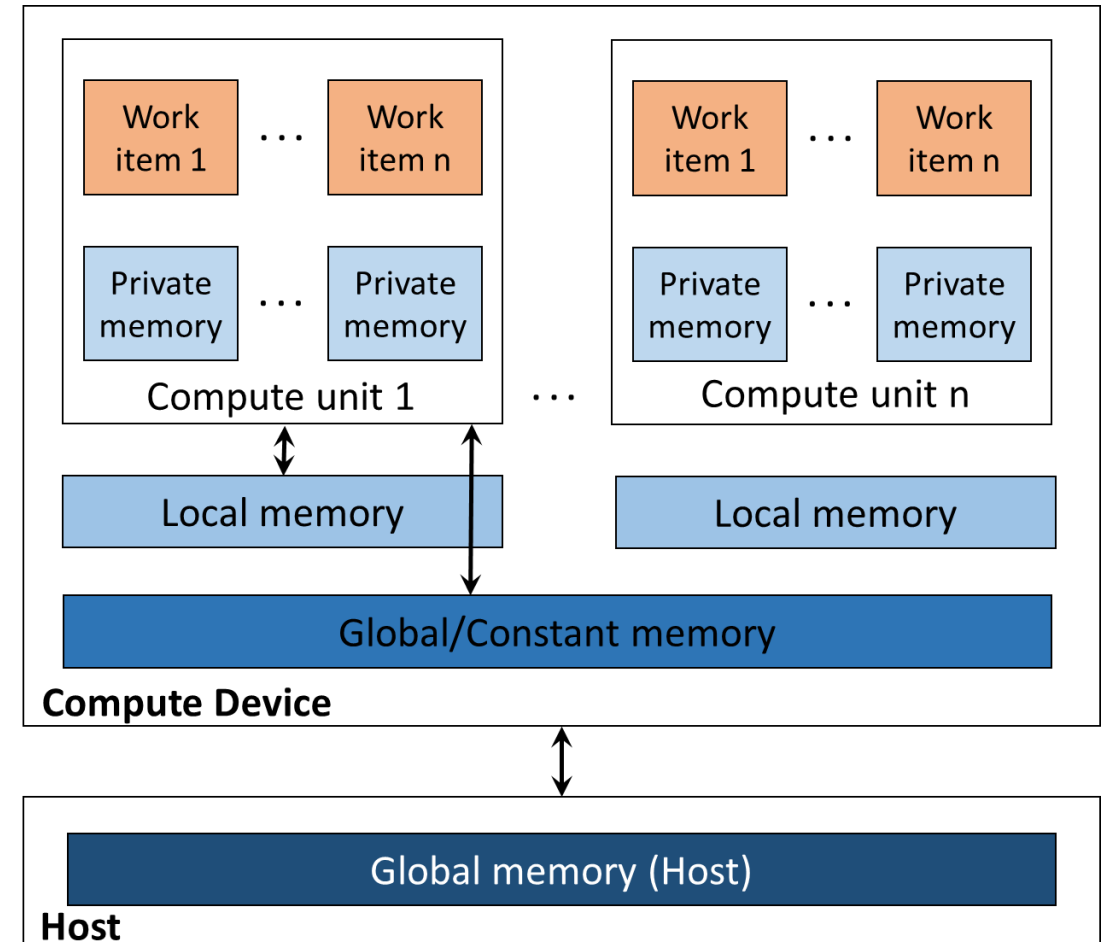
Concurrent data transfer data computation



Time for Host to device data transfer

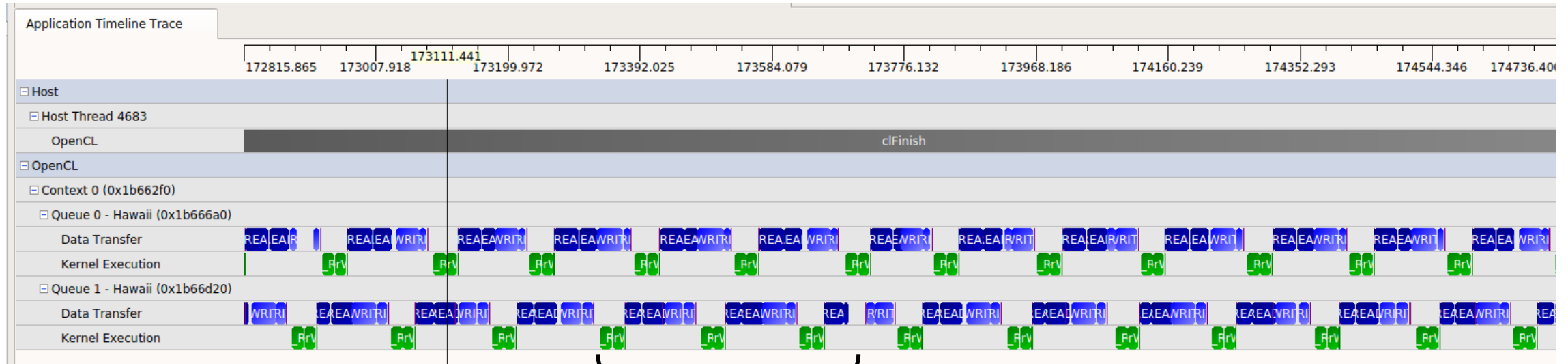
Time for Device to host data transfer

Computation time



OpenCL – Memory hierarchy model

Asynchronous computation and data transfer



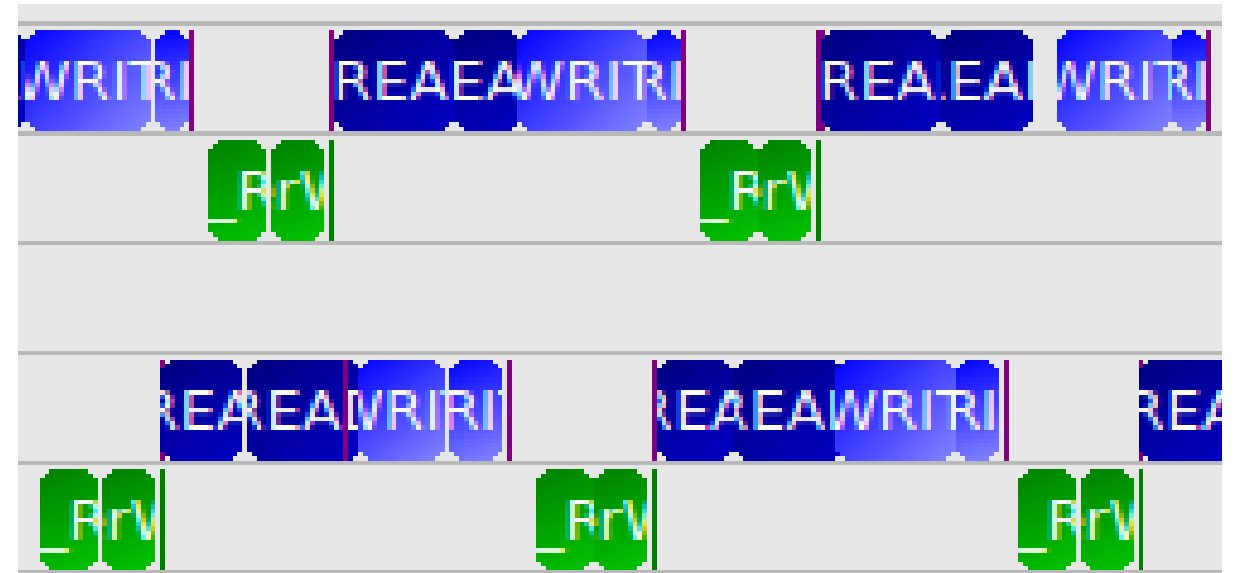
Snapshot of execution timeline in CodeXL

Compute device AMD W8100

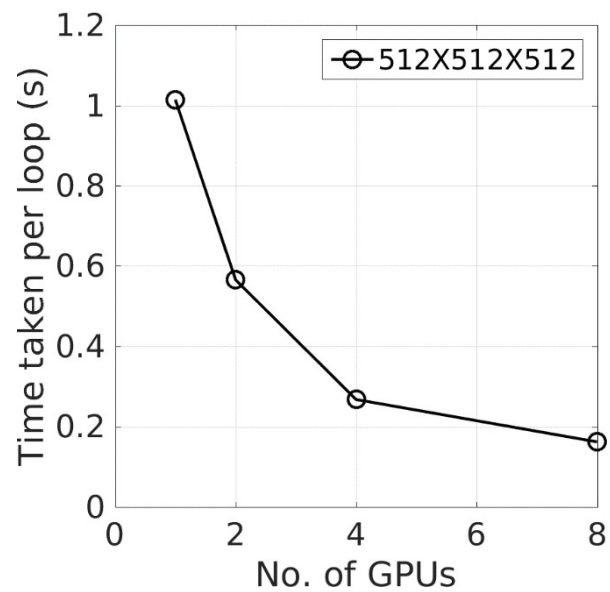
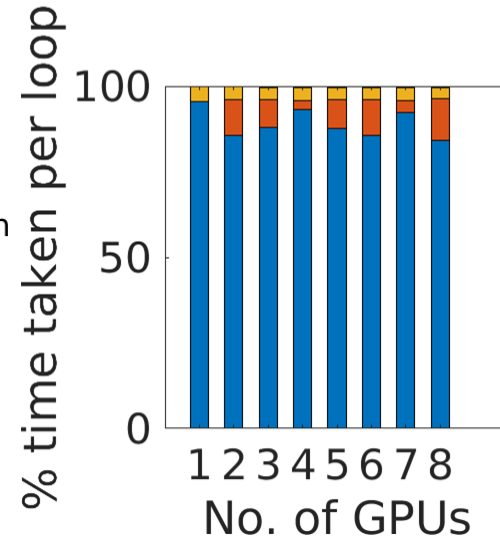
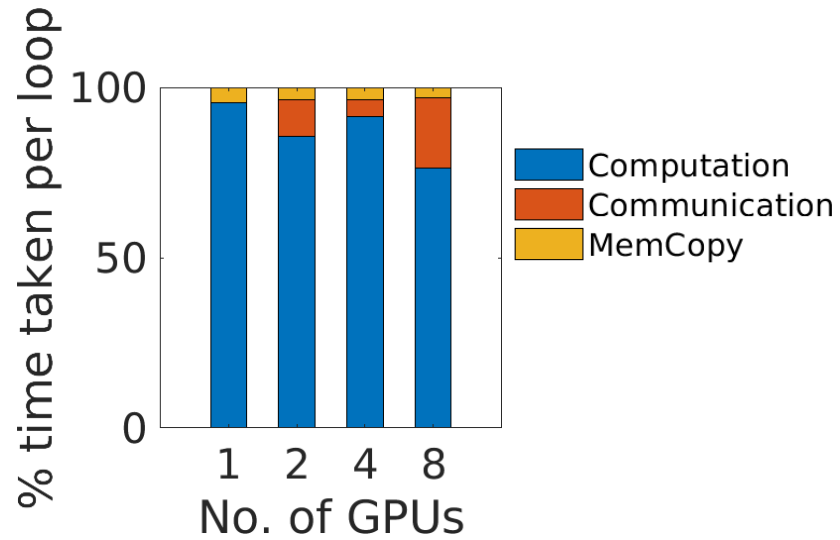
Write to GPU

Read from GPU

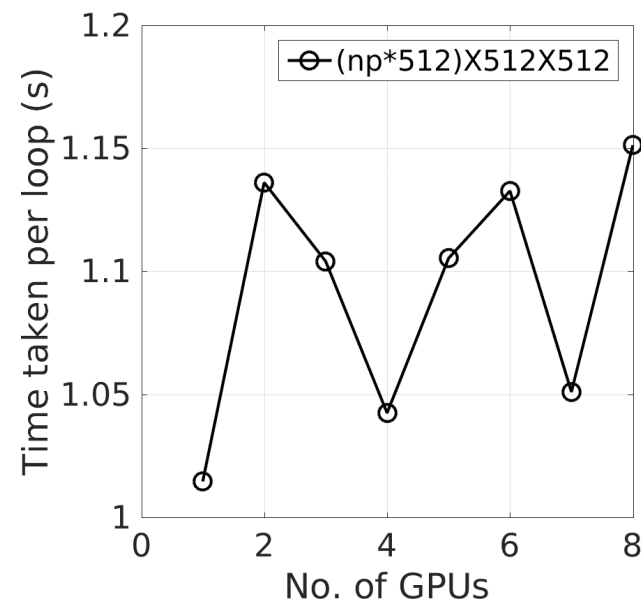
Computation



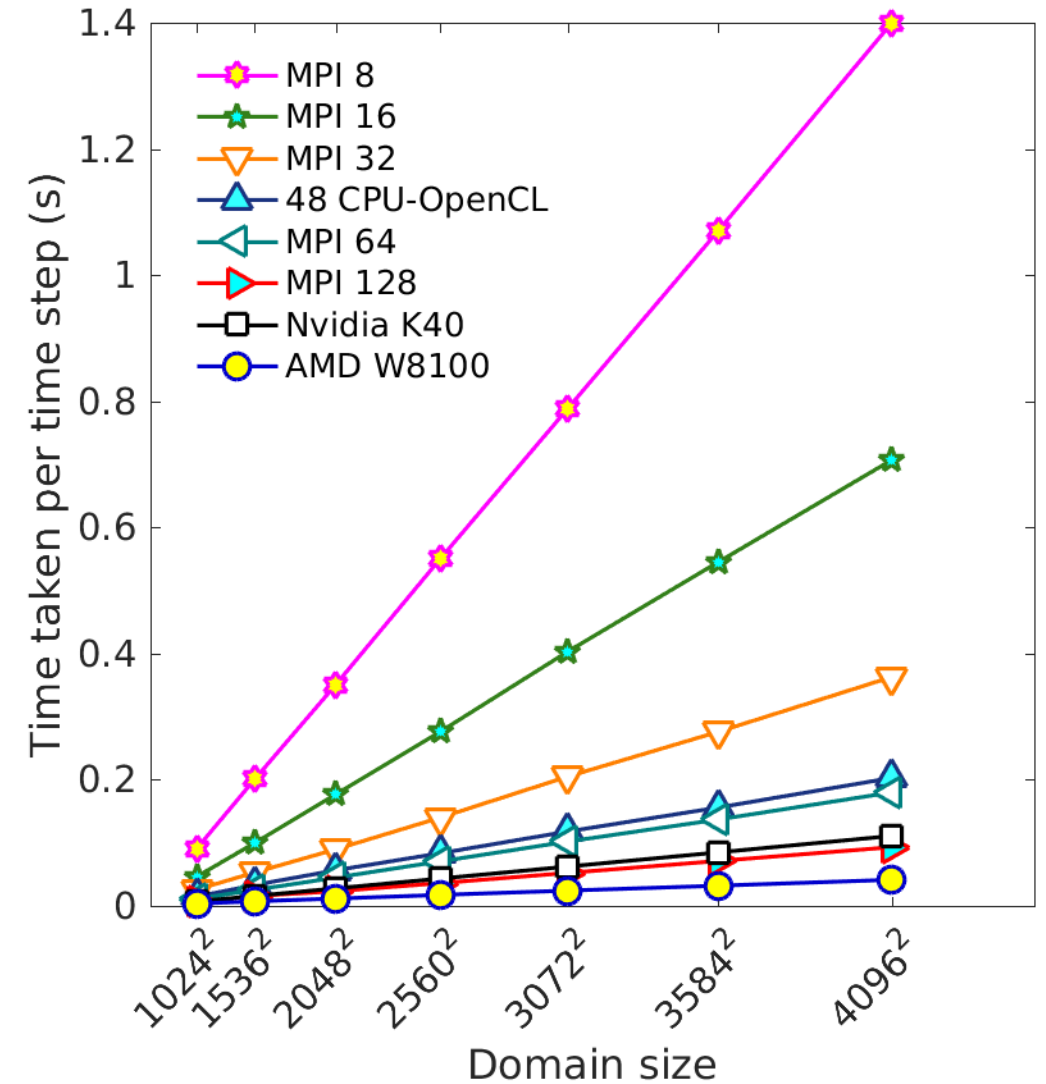
Scaling analysis



Strong scaling

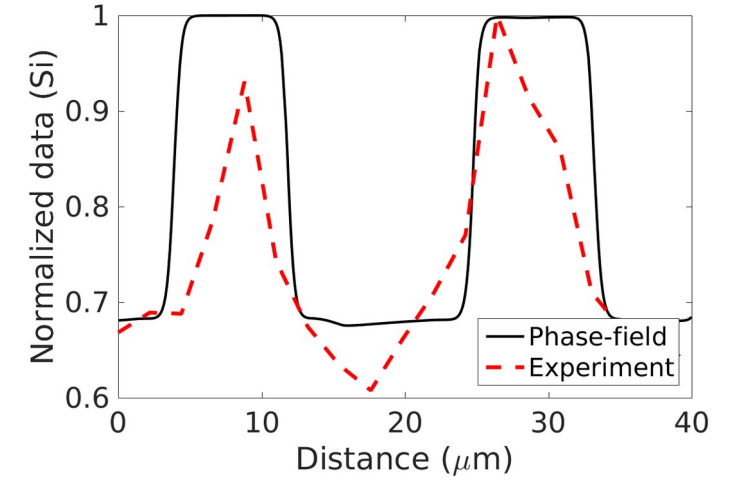
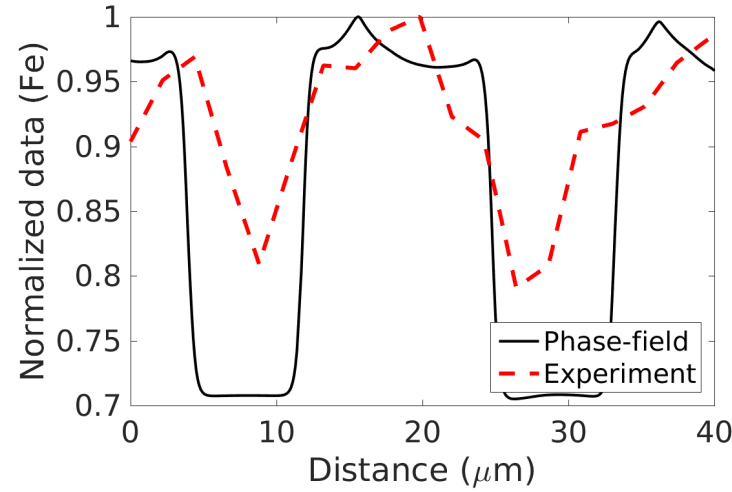
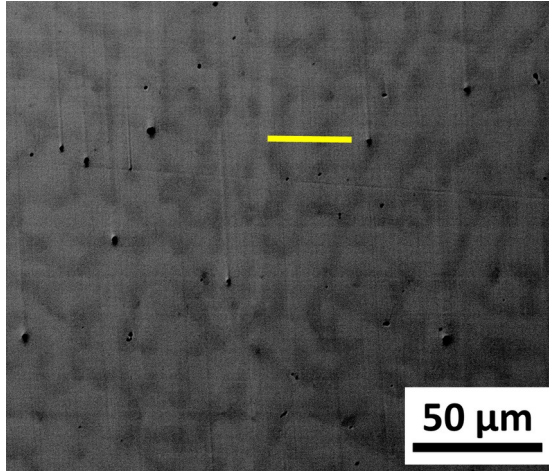


Weak scaling

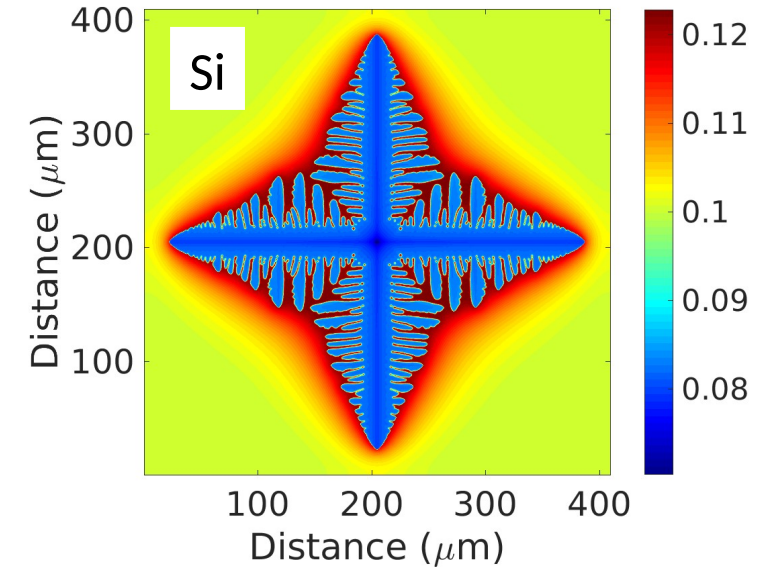
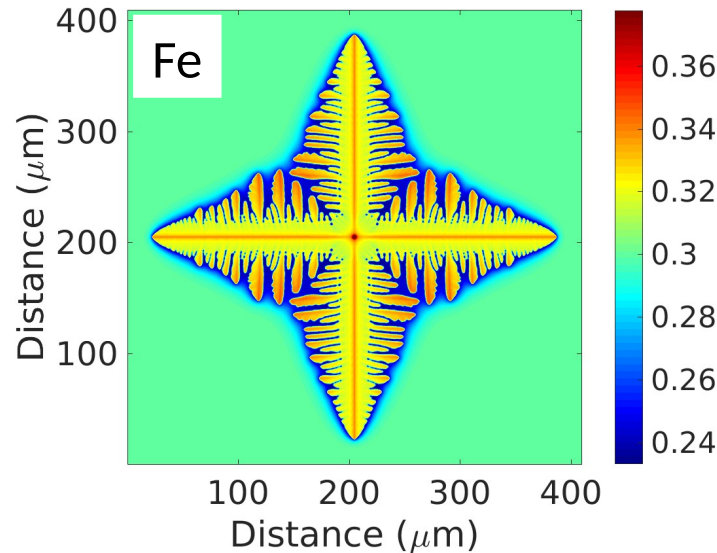
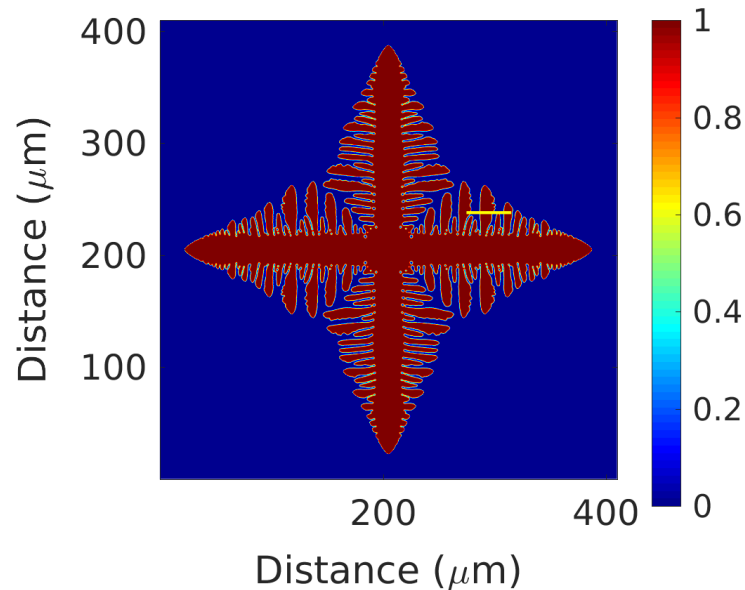


Scaling analysis of the code

- Microstructure and composition profiles



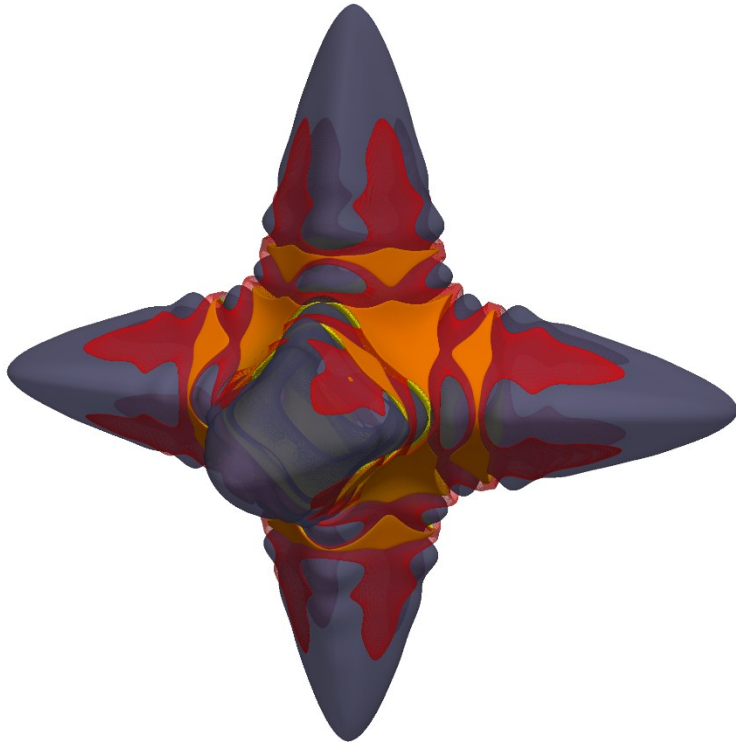
EDS profiles for $\Delta T = 38$ K



Phase-field and composition profiles for $\Delta T = 38$ K

3D dendrite: Ni-Cr-Nb alloy

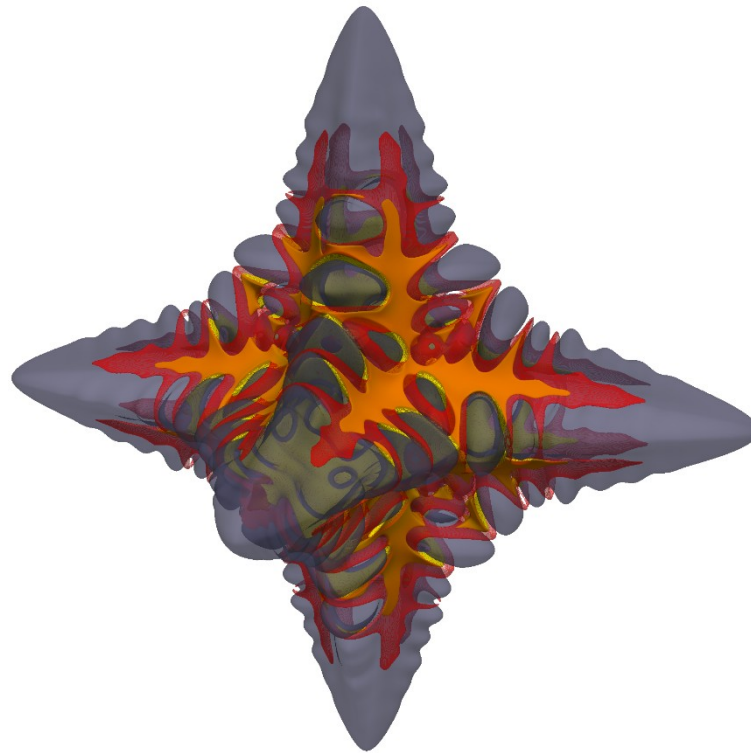
A billion grid points !



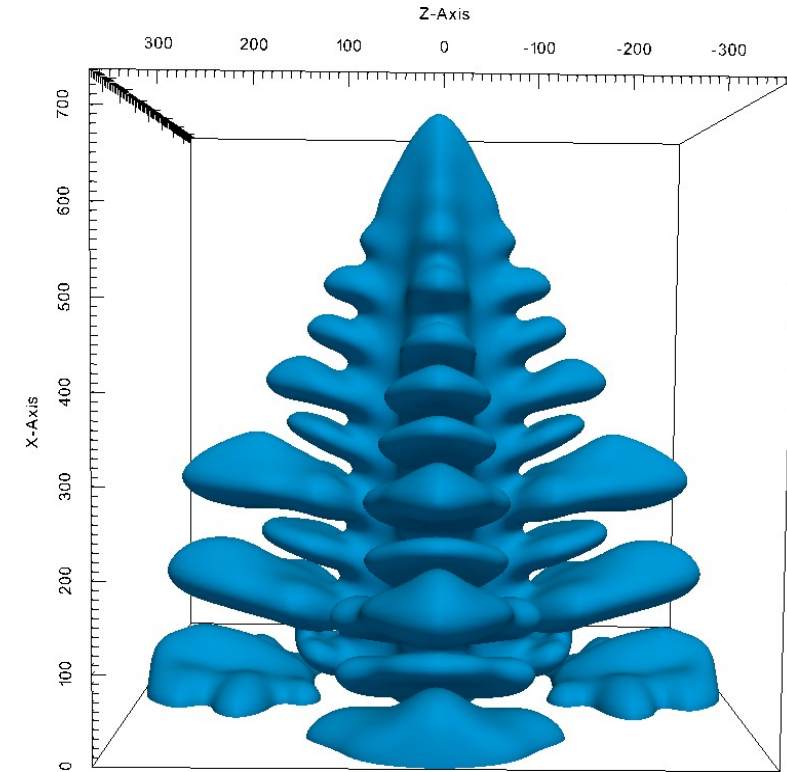
$\Delta T = 6$ K

Phase-field profile with solute contours

$\Delta x = 40.0$ nm. $1024 \times 1024 \times 1024$ grid points



$\Delta T = 8$ K

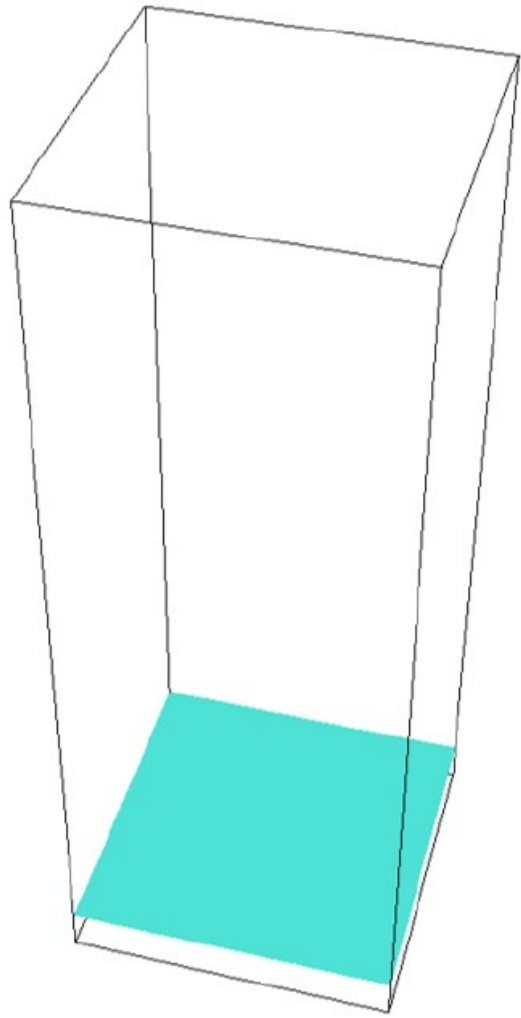


$\Delta T = 8$ K (Moving frame approach)

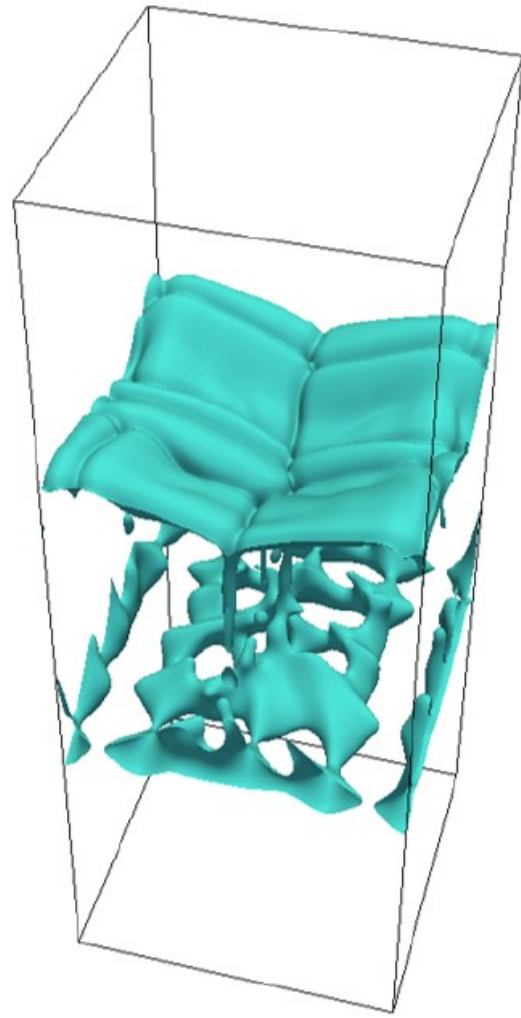
Phase-field profile

$384 \times 384 \times 1024$ grid points

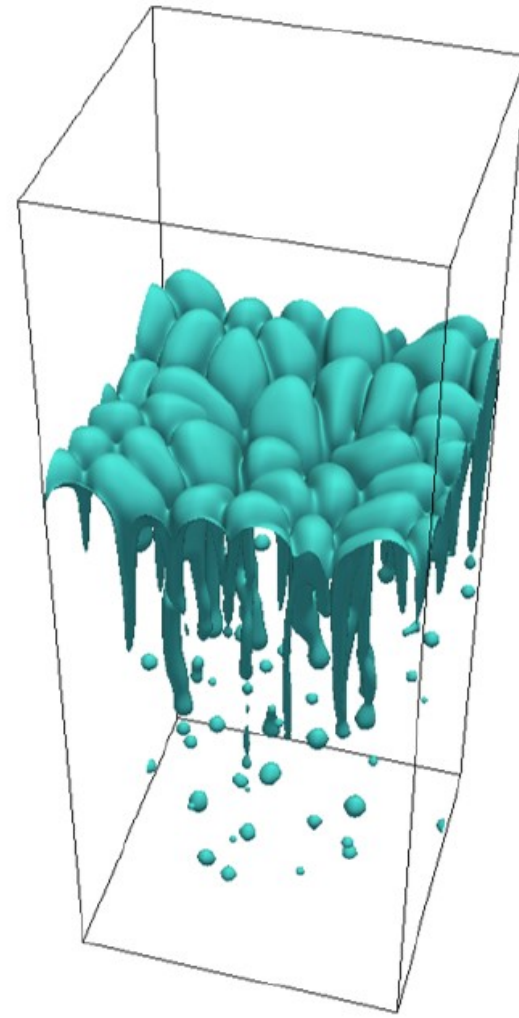
3D simulations



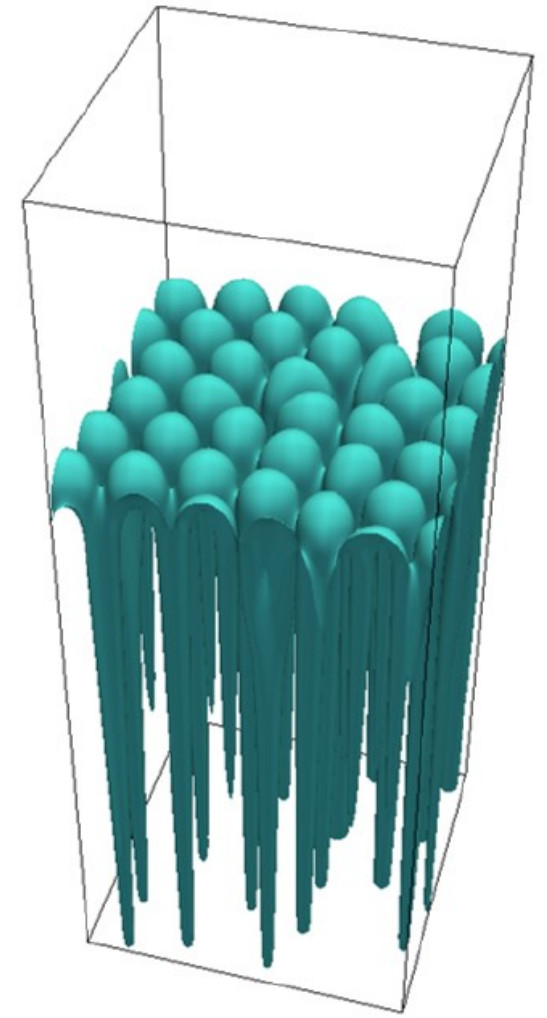
$t = 0 \mu\text{s}$



$t = 101.48 \mu\text{s}$



$t = 118.70 \mu\text{s}$



$t = 514.28 \mu\text{s}$

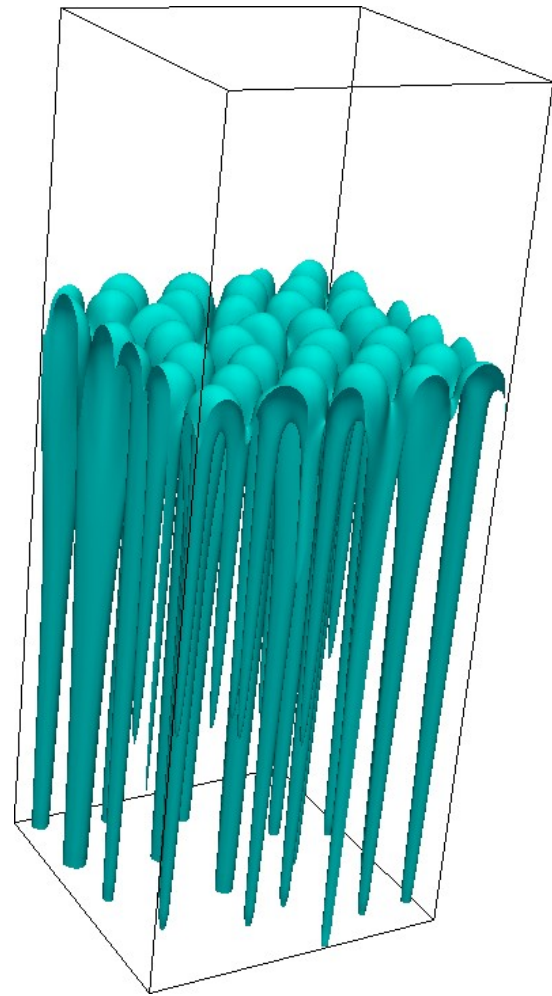
Domain size = $382 \times 382 \times 1024$
= $1.9 \times 1.9 \times 5.1 \mu\text{m}$

$T_G = 5.0 \times 10^6 \text{ K/m}$

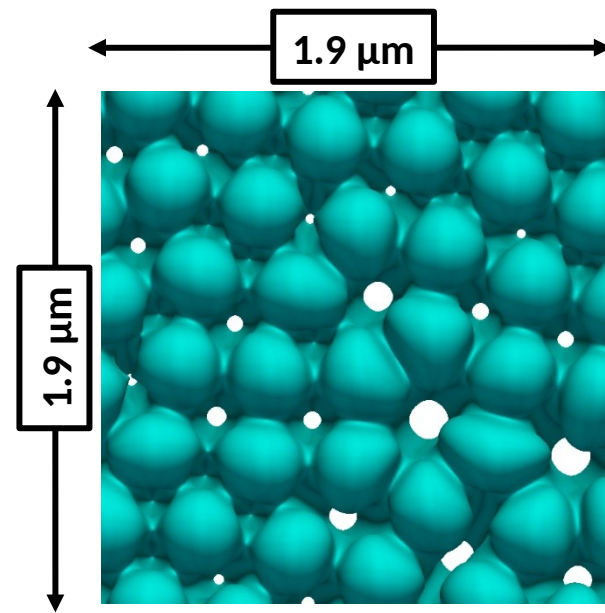
$V_G = 0.1 \text{ m/s}$

$dx = 5 \text{ nm}$

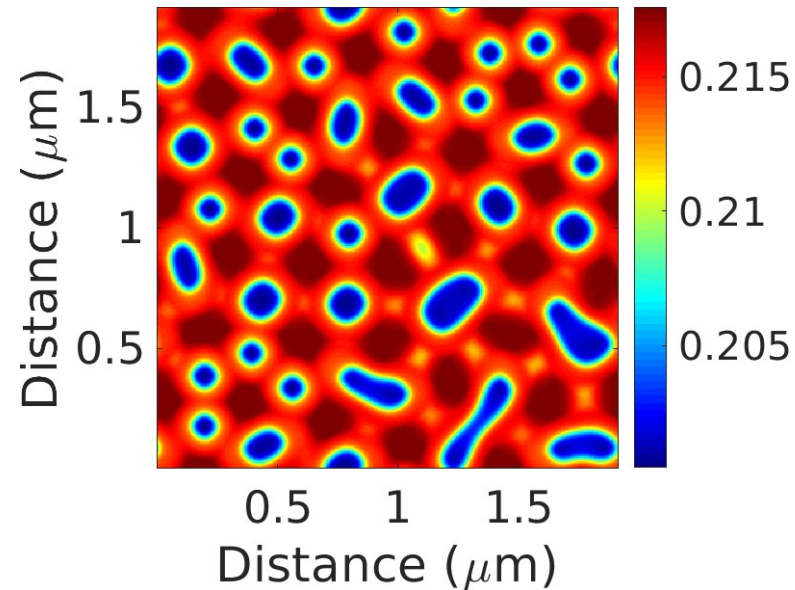
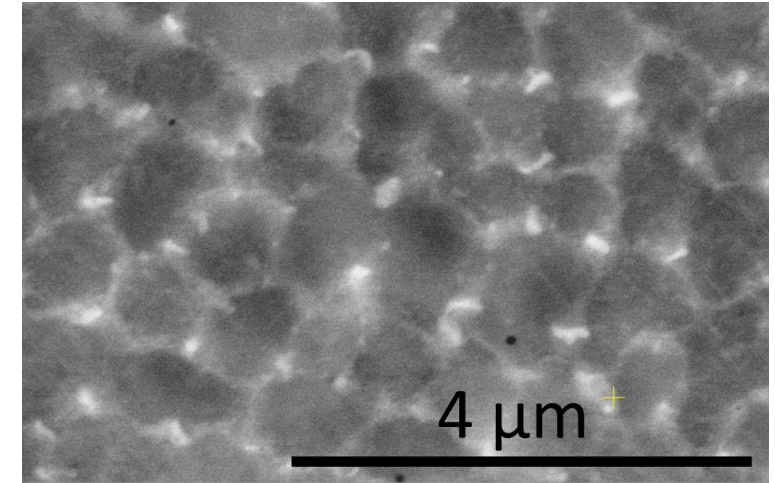
3D simulations



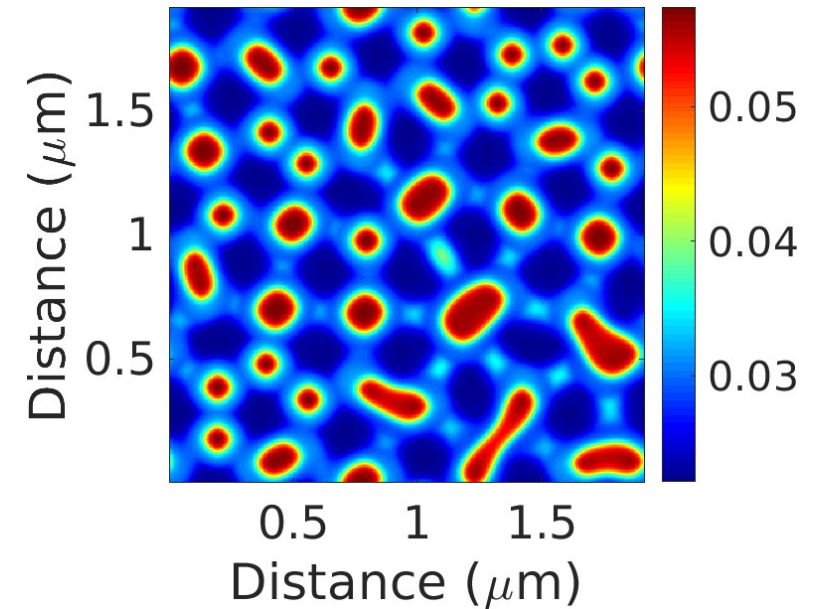
Phase-field contours at $\phi = 0.5$



Top view



Cr



Nb

Domain size = $382 \times 382 \times 1024$
 $= 1.9 \times 1.9 \times 5.1 \mu\text{m}$

$T_G = 5.0 \times 10^6 \text{ K/m}$
 $V_G = 0.1 \text{ m/s}$

$dx = 5 \text{ nm}$

Horizontal cross section of 3D simulation results

Conclusions

- HPC is exciting and useful for real world problems
- Skill sets for HPC are widely applicable
- Large problems can be solved faster using HPC
- HPC is now a publicly available paradigm

Thank you